BookForMe: An Agentic AI-Powered Centralized Booking Platform

Kaavish Report

presented to the academic faculty

by

Muhammad Taqi	mt08073
Muhammad Ahmad Humayun Hanif	mh08072
Jazib Waqas	jw08048
Taha Hunaid Ali	ta08451

In partial fulfillment of the requirements for

Bachelor of Science

Computer Science

Dhanani School of Science and Engineering

Habib University

Spring 2026

BookForMe: An Agentic AI-Powered Centralized Booking Platform

This Kaavish project was supervised by:

_____

Project Supervisor
Faculty of Computer Science
Habib University

0.09

Approved by the Faculty of Computer Science on _____

Acknowledgements

We want to thank the CS faculty at Habib University for their guidance throughout the Kaavish programme, particularly during milestone reviews where their feedback sharpened our system design and evaluation methodology.

We are grateful to the futsal and padel court operators in Karachi who generously shared their WhatsApp booking conversations and allowed us to observe real scheduling workflows. Without their cooperation, our bilingual dataset would not exist.

Finally, we acknowledge the open-source communities behind HuggingFace Transformers, LangGraph, FastAPI, and React Native, whose work made this project technically feasible within one academic year.

0.09

Abstract

Booking recreational and informal services in Pakistan—futsal courts, salons, gaming zones, and beach houses—remains fragmented and inefficient. Customers must individually contact vendors over WhatsApp, wait for manual availability checks, and confirm reservations through unstructured conversation. This process is slow, prone to double-bookings, and poorly suited to users who communicate in Roman Urdu.

BookForMe is a centralized, agentic AI booking platform that addresses these problems. It provides a React Native mobile application for browsing and booking services, and an AI Receptionist that autonomously manages booking requests on the vendor's WhatsApp channel. Core contributions are: (1) a custom bilingual dataset of 657+ annotated utterances in English and Roman Urdu constructed through real vendor conversations and LLM-driven augmentation; (2) a comparative evaluation of six transformer architectures for intent classification, with DistilBERT achieving the best performance of 90.5% accuracy and 91.7% macro-F1; (3) a LangGraph-based cyclic state machine implementing the full booking pipeline with Optimistic Concurrency Control (OCC) in Firestore and OCR-based payment verification; and (4) a social layer with Elo-based matchmaking, community forums, and leaderboards.

A user survey ($n > 50$) found that 72% of respondents rely on WhatsApp or phone calls for bookings and 81% reported frustration with multi-vendor coordination, validating the social relevance of this work. The platform is live at https://jhat-to9p.onrender.com and open-sourced at https://github.com/JazibWaqas/JHAT.

Keywords: Agentic AI, Task-Oriented Dialogue, Bilingual NLU, Roman Urdu, LangGraph, Optimistic Concurrency Control, WhatsApp Business API, DistilBERT.

Contents

List of Figures

List of Tables

1. Introduction

1.1 Problem Statement

Booking recreational and informal services in Pakistan is a deeply fragmented and inefficient process. A customer who wants to reserve a futsal court for Friday evening must: (1) find the vendor's WhatsApp number, (2) send a message asking for availability, (3) wait for a manual reply—sometimes hours later, (4) negotiate a time and price, (5) transfer an advance payment via JazzCash or bank transfer, and (6) send a payment screenshot as confirmation. If the chosen slot was taken by another customer during the wait, the entire cycle restarts with a different vendor.

This friction is not limited to sports courts. Salons, gaming zones, beach houses, and most informal service businesses in Karachi operate identically. No industry-wide booking infrastructure serves this segment. Existing digital solutions are narrow in scope—sports-only apps or salon-specific platforms—and do not reflect the cross-category, bilingual booking behaviour of real users.

A user survey conducted by the team in Spring 2025 ($n > 50$) quantified this friction directly [**survey2025**]:

- **72%** of respondents rely on WhatsApp or phone calls to make bookings.
- **81%** reported frustration with contacting multiple vendors to find an available slot.
- **68%** said they would switch to an app that could handle the conversation and booking on their behalf.

Research by MoldStud [**moldstud2024**] corroborates that automated booking systems "increase efficiency, reduce human error, and improve customer satisfaction." Kumar et al. [**kumar2020**] further show that structured digital workflows reduce errors and missed reservations compared to manual methods. Chatterjee

et al. [**chatterjee2025**] demonstrate through a Turf Booking System study that real-time slot availability and automated conflict resolution improve both user experience and vendor management—but their solution remains domain-specific and does not extend to informal multi-category vendors or support autonomous agent-driven negotiation.

At the vendor side, the problems mirror those of users. Vendors manage incoming WhatsApp requests across multiple simultaneous conversations, maintain schedules in personal notebooks or Google Sheets, and face double-bookings whenever they are offline or slow to reply. Small vendors lack the resources to build dedicated booking systems, causing them to miss revenue and to offer a poor customer experience.

The core computer science problem is therefore: *how can an AI agent understand and autonomously execute bilingual natural-language booking requests in real time, while guaranteeing consistency in a distributed, multi-channel environment?* This decomposes into four sub-problems:

1. **Bilingual NLU:** Parse booking requests in both English and Roman Urdu, handling code-switching ("Mujhe Friday ko 7 baje ke baad paddle court chahiye under 6000 rupees, preferably in defence") and severe orthographic variation (*waqt*, *vakt*, *wkht*, and *tym* are all romanisations of the Urdu word for "time").

2. **Dialogue Management:** Maintain conversation state across asynchronous WhatsApp sessions, handle partial information, manage clarification loops, and route user intent to the correct backend action.

3. **Agentic Execution:** Autonomously check availability, hold slots atomically, verify payment proofs via OCR, and notify vendors—without hallucinating actions or bypassing business rules.

4. **Concurrency Safety:** Prevent race conditions in a distributed NoSQL environment where multiple simultaneous requests may attempt to book the same time slot.

1.2 Proposed Solution

BookForMe is a centralized, agentic AI-powered booking platform that addresses all four sub-problems above. It provides two complementary interaction modes:

1. **React Native Mobile App (Rich Client):** A full-featured mobile application that allows customers to browse vendors by category and location, view real-time availability calendars, select and confirm slots, upload payment proof, and access a social layer (matchmaking, forums, leaderboard).

2. **WhatsApp AI Receptionist (Conversational Client):** An autonomous agent that operates on the vendor's existing WhatsApp Business number. Customers message the vendor in plain English or Roman Urdu; the agent understands their request, checks availability from the Firestore database, holds the slot via Optimistic Concurrency Control, requests a payment screenshot, verifies the amount via OCR, and notifies the vendor for final approval—all without requiring the vendor to participate in the conversation.

Both clients share a single backend and database (the "Single Source of Truth"), meaning that a slot booked via WhatsApp is instantly reflected in the mobile app and on the vendor dashboard. Figure 1.1 gives a high-level overview of this architecture.

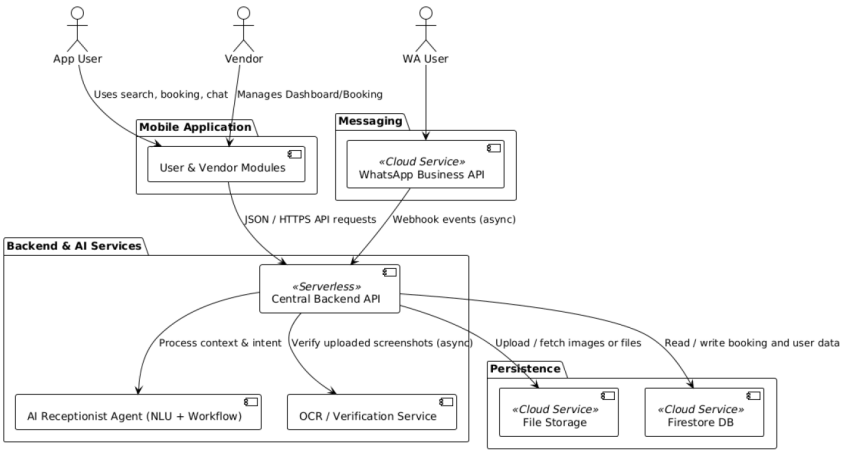

Figure 1.1: High-level overview of the BookForMe platform.

What differentiates BookForMe from existing booking tools is the integration of three capabilities within one platform:

- bilingual NLU tuned to the local mix of English and Roman Urdu,
- a fully autonomous agentic workflow over WhatsApp with transactional safety,
- a social layer for player community and matchmaking across sports, salons, gaming zones, and other informal services.

1.3 Intended User

BookForMe serves three distinct user groups:

App Users (Customers) Residents of Karachi who want to book informal services—predominantly sports-active young adults aged 18–35. They interact with the platform primarily through the React Native mobile app, using the AI chatbot for natural-language search or manual browsing with filters for price, location, and amenities. Social features (player matching, leaderboards, forums) are especially relevant to this group.

WhatsApp Users (Casual Customers) Users who prefer not to install a separate app. They interact exclusively through the vendor's existing WhatsApp Business number. The AI Receptionist handles their entire booking journey—from availability check to payment confirmation—without requiring them to change their communication habits.

Vendors (Small Business Owners) Owners of futsal courts, padel courts, salons, gaming zones, and similar informal services in Karachi. They interact with BookForMe through the Vendor Dashboard, which provides a consolidated booking calendar (aggregating requests from the app and WhatsApp), analytics, pending-booking approvals, and an interface to link their WhatsApp Business account. Crucially, vendors need no technical expertise—onboarding requires only connecting their WhatsApp number.

1.4 Project Gantt Chart and Deliverables

Table 1.1 shows the monthly project timeline across both Kaavish I (CS 491) and Kaavish II (CS 492) semesters.

Table 1.1: BookForMe project timeline and deliverables.

Month	Tasks	Deliverables
Sep–Oct 2025(Kaavish I)	Literature review; user survey; initial SRS; system architecture design; team role assignment; GitHub repository setup.	SRS v1; survey results; architecture diagram; project proposal.
November 2025	SRS v2 and SDS; initial bilingual dataset construction; low-fidelity UI wireframes; Firebase project and Firestore schema setup.	SRS v2; SDS; preliminary dataset (200 utterances); wireframes.
December 2025	Baseline NLU models (intent classification); Firestore CRUD APIs; React Native app skeleton (login, home, vendor listing).	Baseline NLU model; defined DB schema; minimal working booking system.
January 2026(Kaavish II)	Fine-tune NLU pipeline; LangGraph agent implementation; WhatsApp webhook integration; OCC slot locking; OCR payment verification.	Functional AI Receptionist; WhatsApp-to-Firestore data flow; enhanced user dashboard.
February 2026	Vendor dashboard (calendar, analytics, booking approval); social features (matchmaking, forum, leaderboard, chat); model retraining and error analysis.	Full vendor dashboard; active social features; optimised bilingual NLU model.
March 2026	End-to-end integration; unit and integration testing; performance benchmarking; UX testing with real users; mid-evaluation presentation.	Fully integrated platform; test reports; benchmark metrics; mid-eval presentation.
April 2026	Pilot testing with selected Karachi vendors; production deployment on Firebase and Render; final documentation and report.	Production-ready deployment; final report; presentation deck; submission package.

Kaavish I Deliverables:

- Project proposal (approved)

- SRS v1 and v2
- Software Design Specification (SDS)
- Literature review
- Preliminary bilingual dataset
- Low-fidelity wireframes

Kaavish II Deliverables:

- Functional AI Receptionist (WhatsApp agent, live demo)
- Full-stack React Native application (user app + vendor dashboard)
- Fine-tuned DistilBERT NLU model (90.5% accuracy)
- OCR payment verification module
- Social features (matchmaking, forum, leaderboard)
- End-to-end test suite and benchmark results
- Production deployment (https://jhat-to9p.onrender.com)
- This final report

1.5 Key Challenges

Roman Urdu Orthographic Variation Roman Urdu lacks a standardised orthography. A single word like *waqt* (time) may appear as *vakt*, *wkht*, *waqat*, or *tym*, depending on the writer. Standard subword tokenisers (BPE, WordPiece) frequently fail to map these variants to a shared representation. We address this through targeted data augmentation and the use of multilingual models that are exposed to diverse romanisation patterns.

Low-Resource Bilingual Dataset No publicly available labelled dataset exists for English/Roman Urdu booking conversations. We constructed one from scratch through real vendor conversations, manual annotation, and LLM-driven synthetic generation. Starting with a small seed set introduces noise and class imbalance, requiring focal loss training and augmentation strategies.

Asynchronous Conversation State WhatsApp conversations are inherently asynchronous: users may reply after hours, change their mind mid-booking, or send the wrong payment screenshot. Unlike synchronous chatbot frameworks, our agent must persist full conversation state in Firestore and resume gracefully. LangGraph's cyclic state graph design addresses this, but requires careful state serialisation.

Concurrency and Double-Booking Multiple users may simultaneously request the same time slot. In a distributed NoSQL database like Firestore, naïve writes allow race conditions. We implement Optimistic Concurrency Control (OCC) using Firestore atomic transactions with a `hold_expires_at` timestamp, but tuning the hold duration (too short causes false failures; too long ties up inventory) required careful calibration.

LLM Latency Calls to the Gemini API for complex slot extraction add 3–10 seconds of latency per message. The total WhatsApp response time budget (NFR-PERF-03) is 60 seconds. We minimise latency by using the fine-tuned DistilBERT model for fast intent classification (47 ms on CPU) and calling the LLM only for slot extraction and edge cases, caching frequent availability queries.

Prompt Injection Security Malicious users may attempt to trick the WhatsApp agent into bypassing payment requirements through carefully crafted messages. The Neuro-Symbolic architecture—where the LLM classifies intent but all database mutations are executed by deterministic code nodes—prevents such attacks, as the LLM cannot directly issue database commands.

2. Literature Review

This chapter presents the current state of the art across the four research areas that underpin BookForMe: (1) agentic AI systems built on large language models; (2) task-oriented dialogue and state tracking; (3) bilingual and code-switched NLU for South Asian languages; and (4) distributed booking systems with concurrency guarantees. It also establishes the novelty of our work by identifying gaps that existing research does not address and that directly motivate our design choices.

2.1 From Chatbots to Autonomous Agents

2.1.1 Defining the Agentic Architecture

The term "agent" has been used broadly in software engineering and robotics, but its application to large language models (LLMs) represents a specific architectural shift. Wang et al. [**wang2024**] propose a unified framework in which an LLM-based agent comprises four modules: *Profiling* (role and persona definition), *Memory* (short-term context and long-term persistent state), *Planning* (decomposing goals into subgoals and choosing actions), and *Action* (calling external tools and APIs to effect changes in the environment). In this framework the LLM functions as a "Brain" that orchestrates a Perception $\rightarrow$ Reasoning $\rightarrow$ Action cycle, fundamentally distinguishing it from a passive token predictor.

Xi et al. [**xi2023**] further distinguish agents from traditional conversational systems by their capacity to *initiate and manage workflows* rather than merely respond to prompts. A conventional chatbot can answer "Is the 5 PM slot available?"; our agent must answer "Book the 5 PM slot, lock it for ten minutes, verify the advance payment, and notify the vendor." The latter requires stateful read–write operations across multiple external systems—precisely the capability gap this work fills.

2.1.2 Cognitive Architectures: ReAct, PRACT, and Reflexion

Yao et al. [**yao2023**] introduced the **ReAct** (Reasoning + Acting) paradigm, which interleaves chain-of-thought reasoning with external actions, enabling explicit planning traces such as: *Thought*: User requests 5 PM padel. → *Action*: `get_availability("padel","2026-03-15","17:00")`. → *Observation*: Slot available. → *Action*: `create_booking(user, slot)`.

Liu et al. [**liu2024**] extend this with the **PRACT** agent, adding a self-reflection step *before* executing irreversible actions. Before calling `create_booking()`, the agent verifies that the action is consistent with the user's latest stated intent—a critical safeguard where a confirmed booking constitutes a financial commitment.

Shinn et al. [**shinn2023**] demonstrate in **Reflexion** that verbal reinforcement and persistent state across iterations significantly improve agent reliability on multi-step tasks. The agent receives textual feedback from the environment (e.g., "payment amount incorrect") and updates its policy accordingly without gradient updates. This pattern directly informs our OCR rejection loop, where the agent re-prompts the user upon detecting an incorrect payment amount.

2.1.3 Orchestration Frameworks: LangChain and LangGraph

LangChain [**mavroudis2024**] provides a modular abstraction layer that decouples application logic from specific LLM providers, enabling composition of prompt templates, vector stores, and output parsers into unified pipelines. However, standard LangChain implementations rely on Directed Acyclic Graphs (DAGs), which cannot represent the iterative, self-correcting loops required for asynchronous WhatsApp conversations where users may reply late, change their mind, or submit incorrect payment proofs.

LangGraph [**langgraph2024**] addresses this by replacing DAGs with *Cyclic State Graphs*: the agent's behaviour is modelled as a state machine where nodes represent actions and edges encode conditional logic. The framework provides loop

control, state persistence across invocations, and fault recovery— all essential for BookForMe's multi-turn WhatsApp booking conversations. Shang et al. [**shang2024**] confirm in AgentSquare that modular architectures separating Planning, Reasoning, and Memory are substantially more robust for complex tasks than monolithic prompt designs.

2.1.4 Multi-Agent Decomposition

Händler [**handler2023**] proposes a multi-dimensional taxonomy of autonomous LLM-powered multi-agent architectures, showing that complex workflows benefit from assigning distinct roles to specialised sub-agents (intent parsing, verification, database updates, external communications) coordinated by a central orchestrator. Wu et al. [**wu2023autogen**] in AutoGen further argue that effective problem-solving requires cyclic loops—the ability to retry failed actions, request missing information, and self-correct—rather than purely linear pipelines. BookForMe adopts this multi-node, cyclic design in its LangGraph implementation.

2.2 Task-Oriented Dialogue and State Tracking

Task-Oriented Dialogue (TOD) systems have evolved from modular pipelines (NLU → Dialogue State Tracking → Policy → NLG) to end-to-end architectures. **SimpleTOD** [**hosseini2020**] treats the entire dialogue as unified causal language modelling, jointly predicting belief states, system actions, and responses in a single autoregressive model. While effective on standard benchmarks (MultiWOZ), it requires large annotated corpora—a significant barrier for our domain where no pre-built dataset exists.

Shin et al. [**shin2022**] address this data scarcity problem with **DS2** (Dialogue Summaries as Dialogue States), converting conversation history into templated natural-language summaries that map directly to structured slot representations. This summarisation-based approach is particularly useful for short, noisy WhatsApp messages where maintaining a full belief-state vector is impractical.

2.3 Bilingual and Code-Switched NLU

2.3.1 Roman Urdu Challenges

Processing Roman Urdu introduces challenges largely absent from standard multilingual NLP. Bhat et al. [**bhat2023**] classify Roman Urdu as exhibiting severe *orthographic variation*: a single word can be rendered in multiple phonetically plausible romanisations by different writers (e.g., *waqt*, *vakt*, *wkht*, *waqat*, and *tym* all mean "time"). Standard subword tokenisers (BPE, WordPiece) trained on formal corpora frequently fail to map these variants to a shared semantic representation. Integrating a normalisation layer before intent classification is therefore necessary.

2.3.2 Cross-Lingual Transfer and Adapters

Yu et al. [**yu2023**] address cross-lingual dialogue state tracking via contrastive learning, aligning slot embeddings across languages so that semantically equivalent slots in different languages map to nearby vector representations. Wu et al. [**wu2023adapter**] show that parameter-efficient adapter methods can transfer pretrained models to code-switched contexts without full retraining, significantly reducing data and compute requirements.

The Qwen2.5 technical report [**qwen2025**] demonstrates that the 7B variant achieves 79.3 on the MMLU-Multi-Understanding benchmark, the highest among models of comparable size, validating its use as a multilingual backbone.

2.3.3 Multilingual Pretrained Models

Devlin et al. [**devlin2019**] introduced **BERT**, establishing bidirectional masked language modelling as the dominant pretraining paradigm. The multilingual variant (mBERT) was pretrained on 104 languages, including Urdu, providing limited but useful cross-lingual transfer. Conneau et al. [**conneau2020**] demonstrate that **XLM-RoBERTa**, trained on a substantially larger multilingual corpus with improved data filtering, outperforms mBERT on cross-lingual classification. Sanh et al. [**sanh2019**]

show that **DistilBERT**, a distilled 66M-parameter version of BERT, retains 97% of BERT's performance while being 40% smaller and 60% faster—a highly attractive tradeoff for latency-sensitive production deployment.

2.3.4 Data Augmentation for Low-Resource Settings

Given the absence of pre-built bilingual booking datasets, data augmentation is essential. Hou et al. [**hou2020**] show that back-translation and contextual augmentation improve slot-filling F1 by 6–17% and intent accuracy by up to 12% in booking domains with fewer than 500 annotated utterances. An et al. [**an2022**] demonstrate that controllable T5-based generation yields 8–18% absolute gains in intent classification under 10-shot conditions for restaurant and hotel reservation tasks. He et al. [**he2024**] in **SynthDST** show that fully synthetic LLM-generated dialogues can surpass real-data baselines, achieving 4–12% higher Joint Goal Accuracy on MultiWOZ when human annotations are extremely limited. BookForMe leverages LLM-driven paraphrasing and synthetic generation following these findings.

2.4 Retrieval, Planning, and Tool Use

Schick et al. [**schick2023**] demonstrated in **Toolformer** that LLMs can learn to invoke external APIs through self-supervised training, deciding autonomously when and how to call tools—a key capability for our agent's database interactions. Patil et al. [**patil2023**] (**Gorilla**) and Qin et al. [**qin2023**] (**ToolLLM**) extend this to controlled invocation of structured APIs, providing the theoretical basis for our `get_availability()`, `create_booking()`, and `get_services()` tool set.

Lewis et al. [**lewis2020**] introduced **Retrieval-Augmented Generation (RAG)**, conditioning generation on dynamically retrieved documents. In BookForMe, RAG enables the agent to access up-to-date vendor availability and pricing from Firestore rather than relying on memorised parametric knowledge—critical for a real-time booking system. Chen et al. [**chen2025**] present adaptive retrieval

strategies where the agent decides when retrieval is necessary, minimising unnecessary database queries and reducing latency.

2.5 Distributed Booking and Concurrency Safety

Mong'are [**mongare2024**] provides a thorough analysis of idempotency in APIs and distributed systems, specifically addressing the double-booking problem that arises when concurrent requests modify shared mutable state. He recommends idempotent APIs combined with Optimistic Concurrency Control (OCC) or distributed locking as the preferred approach for booking systems. Our implementation directly follows this recommendation, using Firestore atomic transactions with a `hold_expires_at` timestamp to implement OCC.

Singh et al. [**singh2000**] demonstrate the effectiveness of reinforcement learning for dialogue policy optimisation in the NJFun system, working with compressed 62-state state spaces to "greatly reduce" training data requirements—an approach that validates BookForMe's data-efficient learning strategy for asynchronous multi-party interactions.

2.6 Novelty and Gap Analysis

Table 2.1 summarises the specific gaps that BookForMe addresses relative to the existing literature and systems.

Table 2.1: Gap analysis: BookForMe versus existing dialogue and booking paradigms.

System	Focus	Tool Use	Roman Urdu	Async Mem.	Txn. Safety
GPT / General LLM	General chat	✓	✓(partial)	✗	✗
SimpleTOD	Research datasets	✗	✗	✗	✗
ReAct Agents	Web automation	✓	✗	✓(limited)	✗
Domain-specific apps	Single category	✗	✗	✗	✓(partial)
BookForMe	**WA Booking**	✓	✓	✓	✓

The primary research gaps addressed are:

- **Roman Urdu code-switching:** Existing DST and NLU research does not target informal WhatsApp-style Roman Urdu, presenting significant gaps in language modelling and orthographic normalisation.
- **Transactional booking guarantees:** While agent tool-use is well studied, very few works address concurrency, idempotency, and transactional safety in real-world booking systems.
- **Asynchronous conversation flow:** Most TOD frameworks assume synchronous interaction. WhatsApp requires persistent memory and session resumption across arbitrary time delays.
- **Evaluation for informal-economy SMEs:** Agent evaluation literature does not address booking systems in informal economies where reliability, low latency, and no-infrastructure vendor onboarding are essential.

BookForMe synthesises the reviewed literature into a practical LLM-driven booking agent that is novel precisely because it operates at the intersection of all four gaps listed above.

3. Software Requirement Specification (SRS)

3.1 Introduction

3.1.1 Purpose

This document defines the requirements for the **BookForMe** platform. The purpose of BookForMe is to address inefficiencies and fragmentation in booking informal, local services (such as sports courts, salons, and gaming zones) within Karachi. The system replaces unstructured communication and manual record-keeping with a centralized, intelligent, and reliable platform.

It provides a streamlined booking interface for end-users alongside a social community layer for players. Vendors receive tools to manage bookings and availability across all channels, including an autonomous AI assistant on WhatsApp, reducing operational friction and revenue loss from missed or mishandled requests.

3.1.2 Scope

The BookForMe system comprises four primary components operating together:

1. **User-facing mobile application:** A React Native (Expo) app through which customers discover venues, view real-time slot availability, place bookings, and participate in social features including friends, direct messaging, forums, matchmaking, and a ranked leaderboard.

2. **Vendor dashboard:** A vendor-facing React Native interface to register, manage a business profile (services, pricing, operating hours), view a consolidated booking calendar with live slot status, approve or reject pending payments, and connect a WhatsApp Business account.

3. **AI agent backend:** The intelligent engine built in Python (FastAPI and LangGraph). It receives WhatsApp traffic via webhooks, performs bilingual (Roman Urdu/English) natural language understanding using Groq (Qwen 3 32B), manages stateful conversational booking workflows, enforces concurrency control via Firestore transactions, and coordinates payment proof and vendor approval flows.

4. **Admin control panel:** A platform-level administrative interface for platform governance: vendor registration approval and moderation, slot generation, stale lock release, and oversight of pending payments across all vendors.

3.2 Functional Requirements

3.2.1 Module: User (Booking & Discovery)

- `FR-CUST-01`: The system shall allow users to browse and search registered vendors by service category (e.g., Padel, Futsal, Cricket) and location area.
- `FR-CUST-02`: The system shall provide filtering options (e.g., price, area, sport type).
- `FR-CUST-03`: The system shall display vendor profiles including name, location, services offered, and pricing.
- `FR-CUST-04`: The system shall present a real-time live court grid showing available, held, and booked slots per court per time block.
- `FR-CUST-05`: The system shall allow users to select an available time slot and confirm a booking.
- `FR-CUST-06`: The system shall confirm successful bookings to the user via the app interface.

3.2.2 Module: User (AI Chat & Discovery)

- `FR-CUST-07` (Smart filter): The system shall provide an in-app AI chat interface that parses natural-language queries in English or Roman Urdu (e.g., "Aaj Raat DHA mei konsa padel available hei under 2000") and returns filtered venue results with available slots.
- `FR-CUST-08`: The in-app AI assistant shall return matching venues, available slot times, and pricing in a single response.

3.2.3 Module: User (Social & Community)

- `FR-SOC-01`: The system shall provide user registration and login with role selection (Customer or Vendor).
- `FR-SOC-02`: The system shall allow users to create and manage a public profile.
- `FR-SOC-03`: The system shall provide a friend system: send, accept, and deny friend requests.
- `FR-SOC-04`: The system shall allow users to send and receive private direct messages with friends.
- `FR-SOC-05`: The system shall provide a community forum where users can create, view, like, and comment on public posts.

3.2.4 Module: User (Matchmaking & Ranking)

- `FR-MATCH-01`: The system shall allow users to create and join open matches (Casual or Ranked) for sports such as Padel, Futsal, and Cricket.
- `FR-MATCH-02`: The system shall calculate and update user Elo ratings based on ranked match outcomes.
- `FR-MATCH-03`: The system shall display a public leaderboard of top-ranked players, filterable by sport, showing rank, points, and tier (e.g., Silver).

3.2.5 Module: Vendor

- `FR-VEND-01`: The system shall provide secure registration and login for vendors, subject to admin approval before activation.
- `FR-VEND-02`: Vendors shall be able to create and manage their business profile (services, pricing, operating hours).
- `FR-VEND-03`: Vendors shall be able to view a live court grid calendar showing all bookings by court, time, and booking source (WhatsApp or walk-in).
- `FR-VEND-04`: The vendor dashboard shall display daily revenue, bookings count, pending action items, and upcoming unbooked slots.
- `FR-VEND-05`: The system shall provide an interface for vendors to generate availability slots and manage bookings.
- `FR-VEND-06`: The vendor dashboard shall provide an interface to review and approve or reject pending payments.
- `FR-VEND-07`: The system shall allow vendors to connect their WhatsApp Business account to activate the AI receptionist.

3.2.6 Module: AI Agent (WhatsApp Receptionist)

- `FR-AGENT-01`: The system shall receive inbound WhatsApp messages via a configured Meta webhook.
- `FR-AGENT-02`: The system shall send outbound WhatsApp replies (slot listings, confirmations, rejections) via the WhatsApp Business API.
- `FR-AGENT-03`: The system shall process inbound messages in English and Roman Urdu to extract booking intent and entities (service, date, time, area).
- `FR-AGENT-04`: The system shall maintain full conversational context (dialogue state) for ongoing WhatsApp interactions, persisted in Firestore across delays.

- `FR-AGENT-05`: Based on NLU output and real-time availability, the system shall present matching slots to the user and accept confirmation.
- `FR-AGENT-06`: The system shall implement Optimistic Concurrency Control via Firestore transactions to prevent double-booking across all channels simultaneously.
- `FR-AGENT-07`: After a user confirms a booking, the system shall request payment proof (screenshot) and set the booking to a hold state with an expiry window.
- `FR-AGENT-08`: Upon receiving the screenshot, the system shall update the booking to pending approval, attach the image, and notify the vendor dashboard.
- `FR-AGENT-09`: Upon vendor approval, the system shall update the booking status to confirmed and send a confirmation message with a booking reference to the user on WhatsApp.

3.2.7 Module: Admin

- `FR-ADMIN-01`: The system shall provide a platform administrator interface displaying pending vendors, pending payments, locked slots, and total active vendors.
- `FR-ADMIN-02`: The system shall allow admins to approve, suspend, or delete vendor accounts.
- `FR-ADMIN-03`: The system shall allow admins to generate availability slots across all vendors for a configurable window (e.g., next 14 days).
- `FR-ADMIN-04`: The system shall allow admins to release stale slot locks caused by abandoned booking sessions.

3.2.8 Use Case Diagrams

Figure 3.1 shows the high-level system overview. The use case interactions are summarized below:

- **Customer** uses the mobile app to log in, discover venues, book slots, use the AI chat filter, and access social features (forum, friends, matches, leaderboard).
- **WhatsApp user** interacts with the AI receptionist: sends booking requests in English/Roman Urdu, confirms a slot, submits payment proof, and receives a confirmation.
- **Vendor** uses the vendor dashboard to manage their profile, view the live court calendar, handle payment approvals, and link their WhatsApp account.
- **Admin** uses the admin panel to approve vendors, manage slots, and oversee payments.

3.3 Non-Functional Requirements

3.3.1 Performance

- `NFR-PERF-01`: Key app screens (vendor search, live court grid) shall load within 30 seconds on a stable mobile connection.
- `NFR-PERF-02`: The AI agent NLU processing for a WhatsApp message shall complete within 60 seconds.
- `NFR-PERF-03`: Total time from receiving a WhatsApp availability query to sending the initial response shall not exceed 60 seconds.
- `NFR-PERF-04`: The WhatsApp webhook shall handle at least four simultaneous incoming requests without failure.

3.3.2 Security

- `NFR-SEC-01`: All user and vendor access shall be protected via Firebase Authentication.

- NFR-SEC-02: All external API keys (WhatsApp, cloud AI providers) shall be stored as environment variables, not hardcoded.
- NFR-SEC-03: The WhatsApp webhook shall verify all incoming requests using the Meta verify token.
- NFR-SEC-04: All data transmission between clients, the backend, and external APIs shall use HTTPS/TLS.

3.3.3 Reliability

- NFR-REL-01: The system shall target 99.0% uptime during peak operating hours.
- NFR-REL-02: The AI agent shall handle external API errors gracefully, logging them without crashing the main service.

3.3.4 Scalability

- NFR-SCAL-01: The backend shall support scaling via cloud deployment to handle increased load.
- NFR-SCAL-02: The Firestore schema shall support at least 100 concurrent users without significant degradation.
- NFR-SCAL-03: The system shall support at least 10 active vendors without degradation.

3.3.5 Usability

- NFR-USAB-01: A new user shall be able to complete a booking in fewer than eight primary steps from the home screen.
- NFR-USAB-02: Key vendor dashboard functions (calendar, pending approvals) shall be accessible within six taps from the main dashboard.

- `NFR-USAB-03`: WhatsApp agent responses shall be clear and appropriate in both English and Roman Urdu.

3.4 System Block Diagram

Figure 3.1 presents the high-level system block diagram for the BookForMe platform, showing the frontend layer (customer app, vendor dashboard, admin panel), the backend API and AI agent layer, and external services (WhatsApp Business API, Firebase, Firestore).

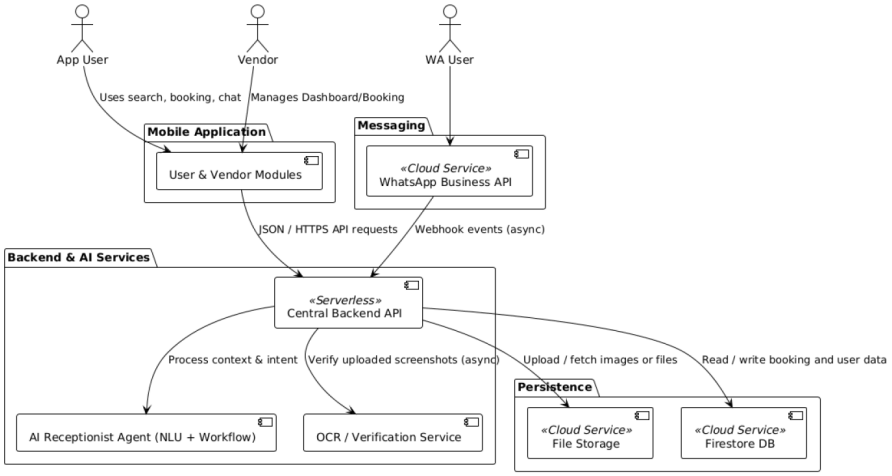

Figure 3.1: System block diagram for the BookForMe platform.

3.4.1 Component descriptions

- **Customer mobile app** — React Native (Expo). Handles discovery, booking, AI chat filter, and all social features. Communicates with the backend via HTTPS APIs.
- **Vendor dashboard** — React Native. Profile management, live court calendar, pending payment review, and WhatsApp account linking.

- **Admin panel** — React Native. Platform governance: vendor approval/moderation, slot generation, stale lock release, payment oversight.
- **AI agent backend** — Python/FastAPI with LangGraph. Manages the WhatsApp webhook conversation loop, bilingual NLU via Groq, stateful booking workflow, and Firestore-backed concurrency control (OCC).
- **Cloud Firestore** — Central NoSQL store for profiles, slots, bookings, conversation state, social data, and payment records.
- **WhatsApp Business API** — Inbound webhooks and outbound messaging channel for the AI receptionist.
- **Firebase Authentication** — Identity management for customers, vendors, and admins.

3.5 System Screenshots

The following screenshots from the deployed BookForMe application illustrate the key user-facing and vendor-facing interfaces described in the functional requirements above.

3.5.1 Authentication

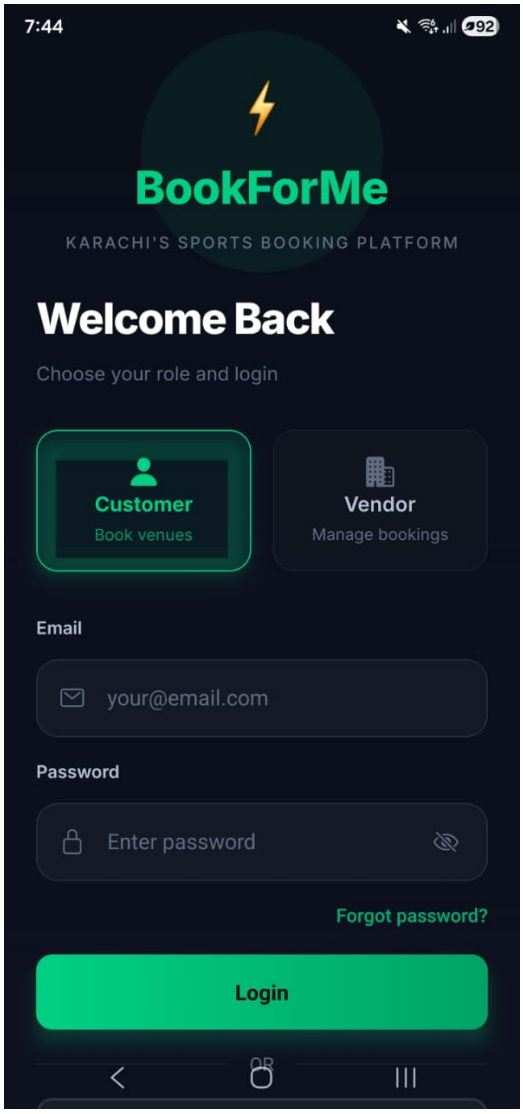

Figure 3.2: Login screen with role selection (Customer or Vendor).

3.5.2 Customer application

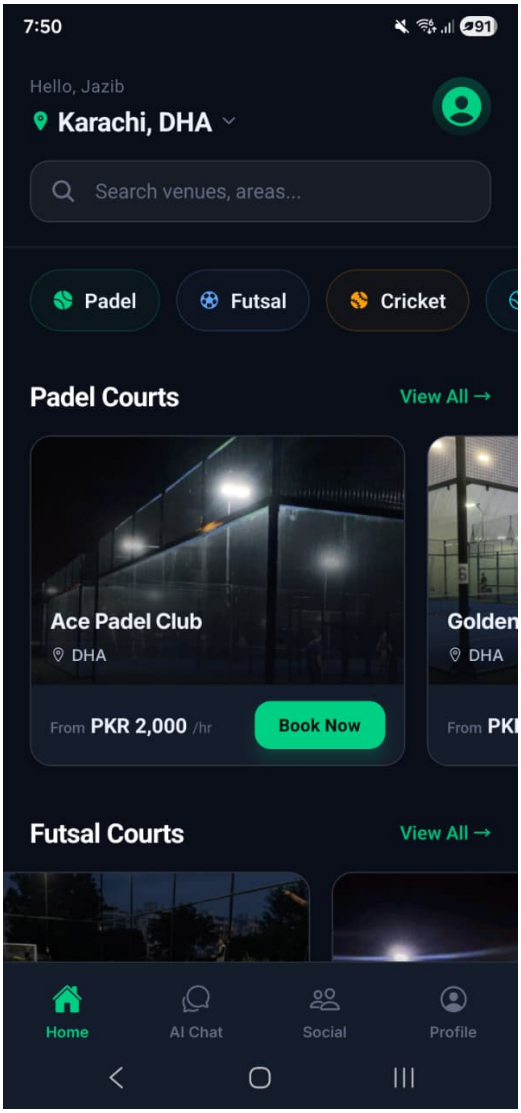

Figure 3.3: Customer home: venue discovery by sport and area.

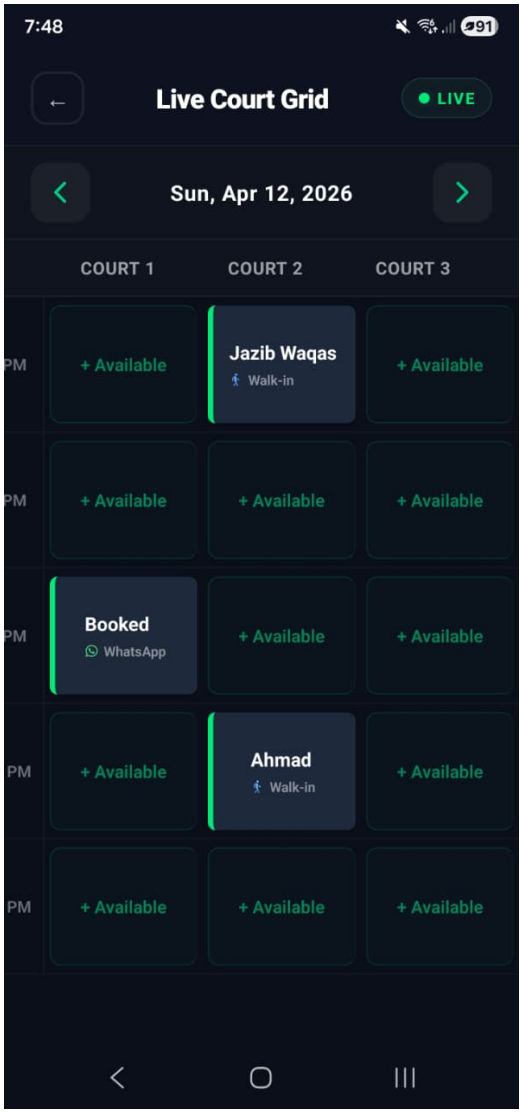

Figure 3.4: Live court grid showing real-time slot availability and booking source.

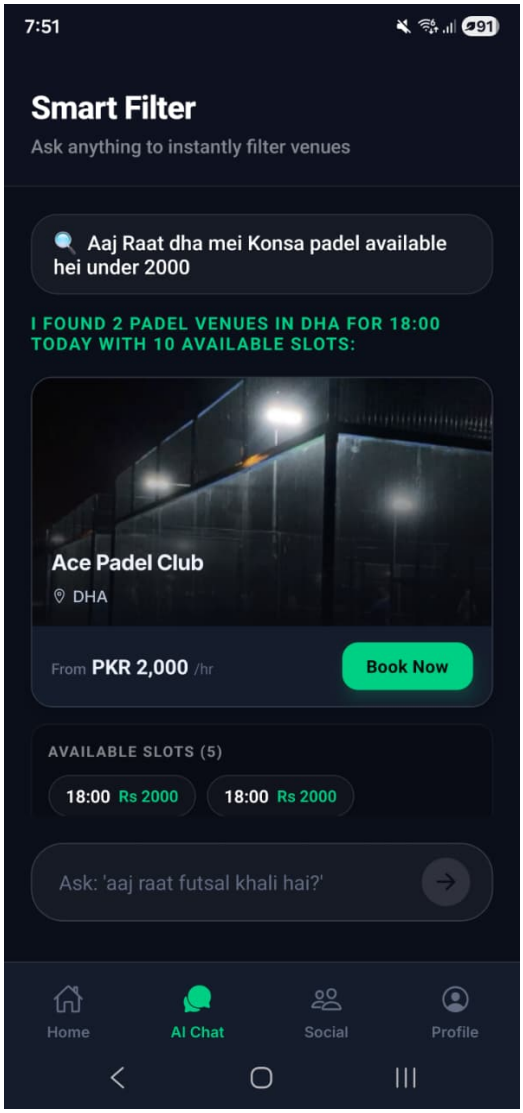

Figure 3.5: In-app smart filter: AI chat parses Roman Urdu/English queries and returns filtered venues with available slots.

3.5.3 Social hub

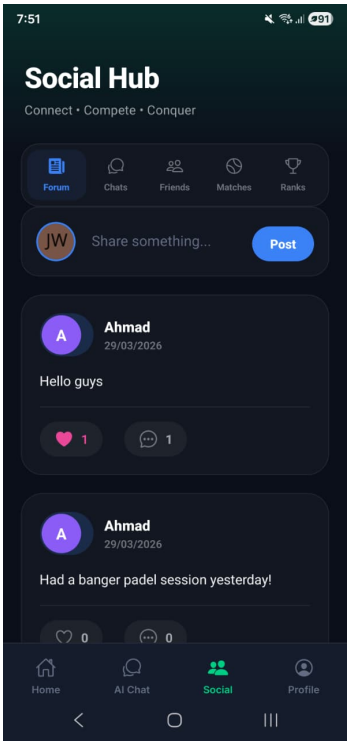

Figure 3.6: Community forum.

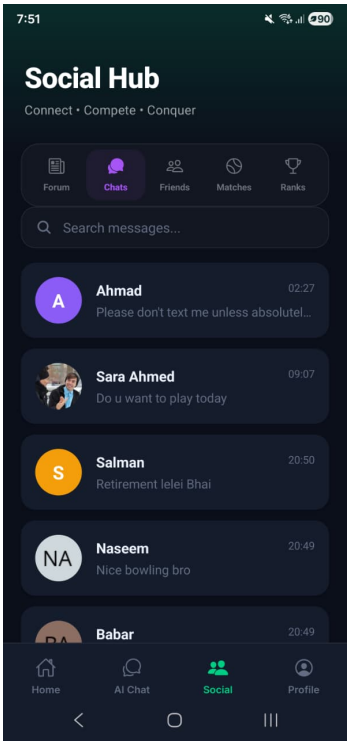

Figure 3.7: Direct messages.

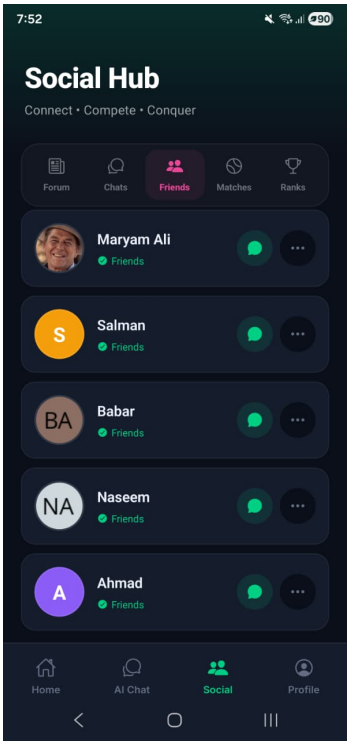

Figure 3.8: Friends list.

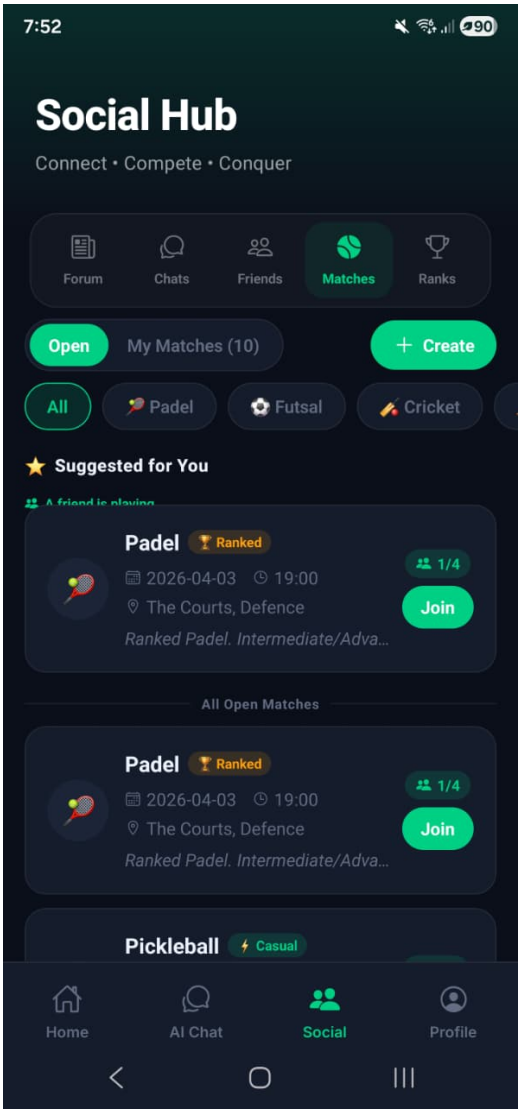

Figure 3.9: Open matches: casual and ranked lobbies.

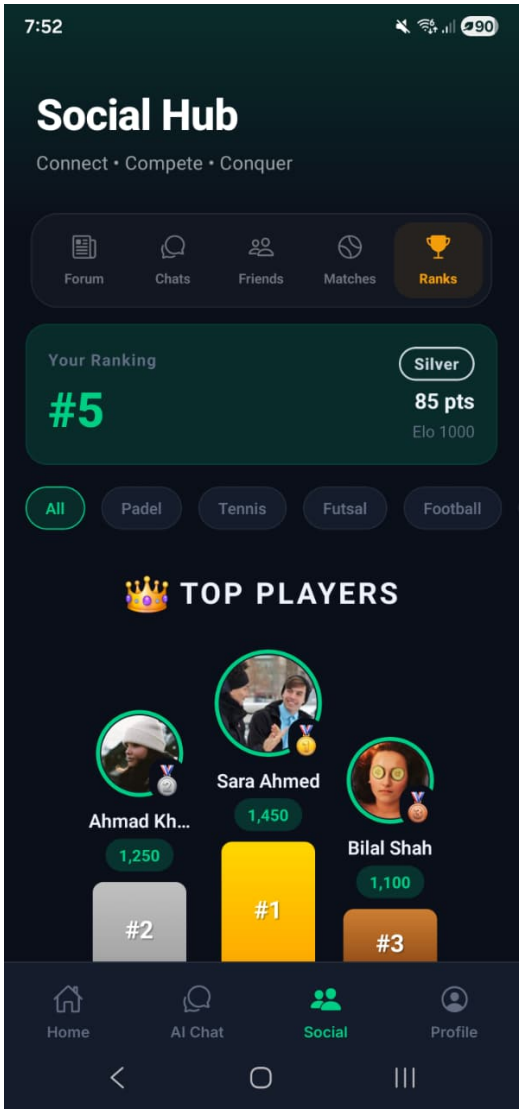

Figure 3.10: Ranked leaderboard with Elo ratings and tiers.

3.5.4 Vendor dashboard

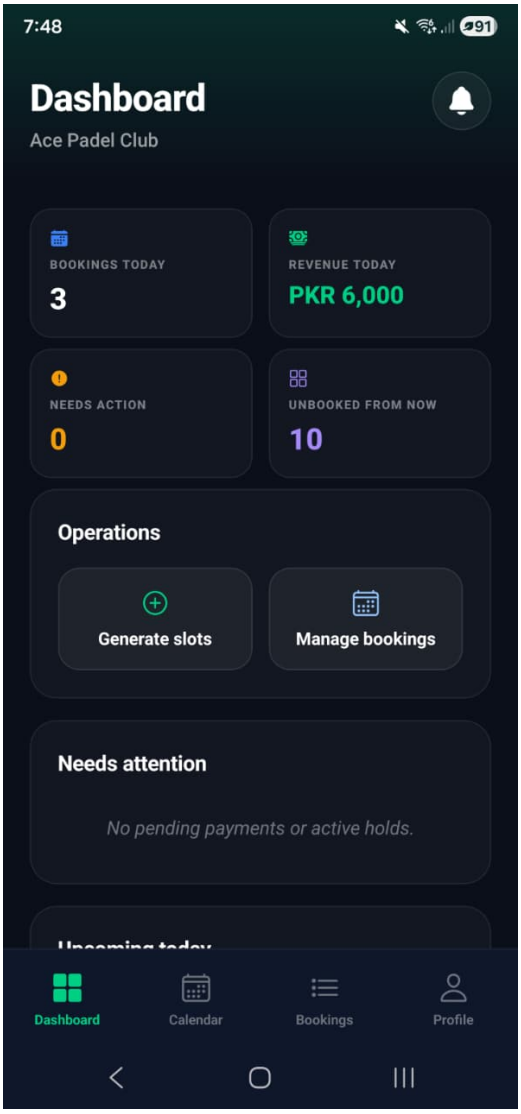

Figure 3.11: Vendor dashboard: daily stats, pending actions, slot operations, and upcoming bookings.

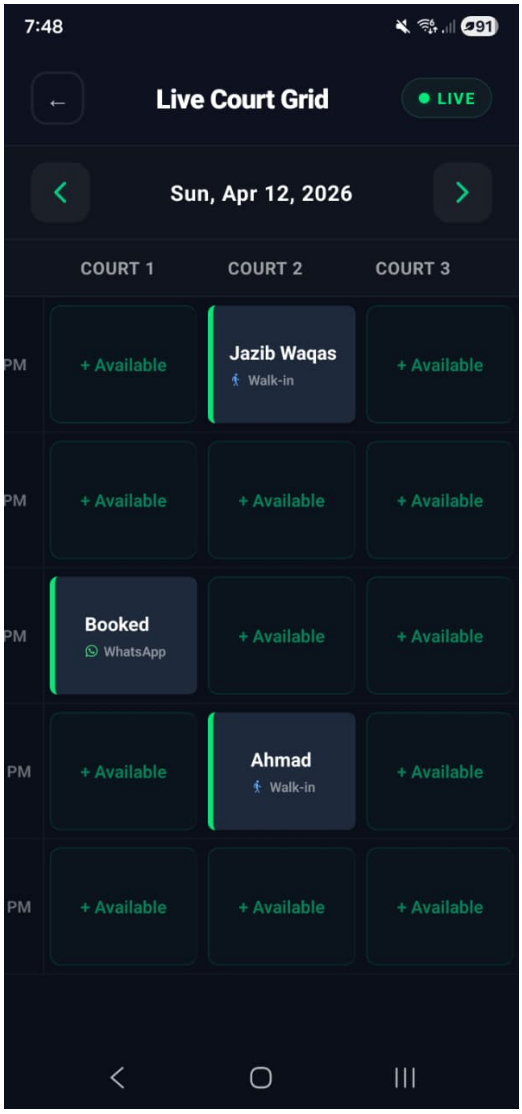

Figure 3.12: Live court grid calendar: per-court slot status with WhatsApp and walk-in booking sources.

3.5.5 AI receptionist (WhatsApp)

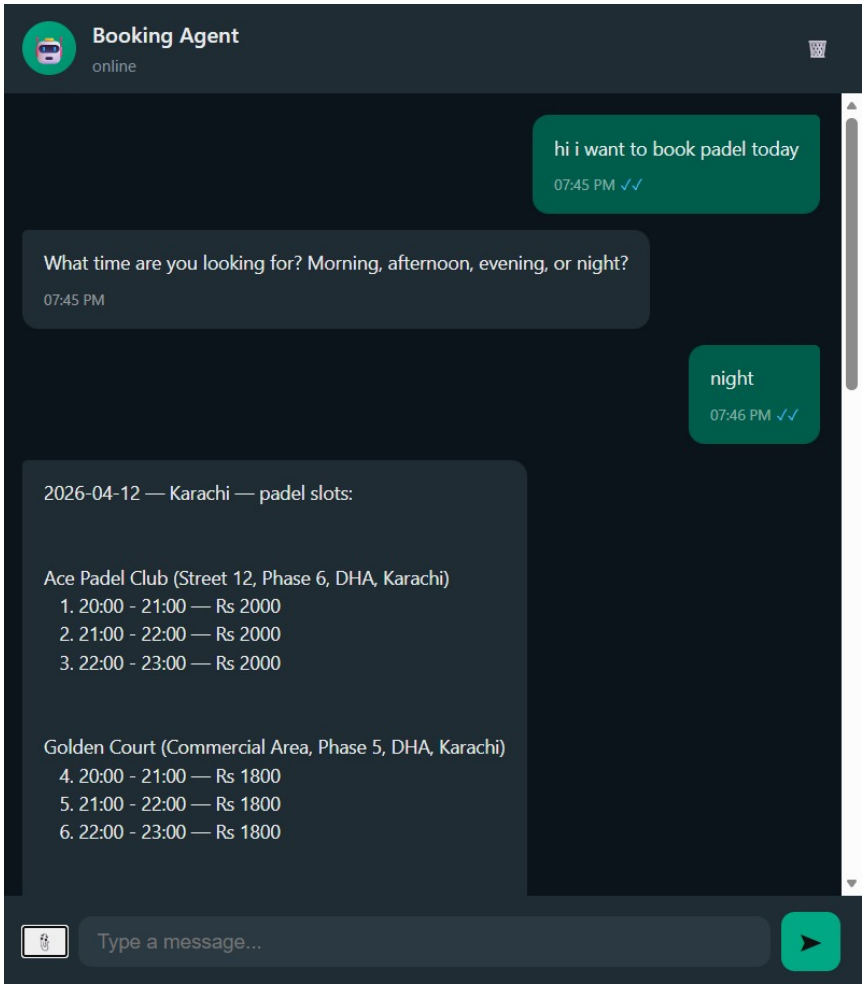

Figure 3.13: WhatsApp AI receptionist: bilingual booking conversation returning available padel slots across multiple venues.

3.5.6 Admin control panel

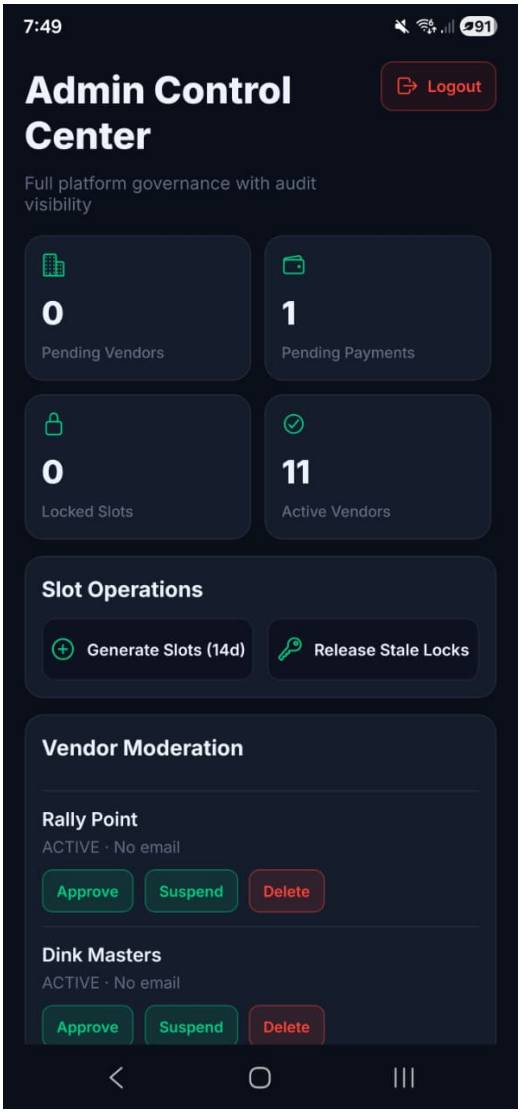

Figure 3.14: Admin control center: platform-wide stats, slot generation, and vendor moderation.

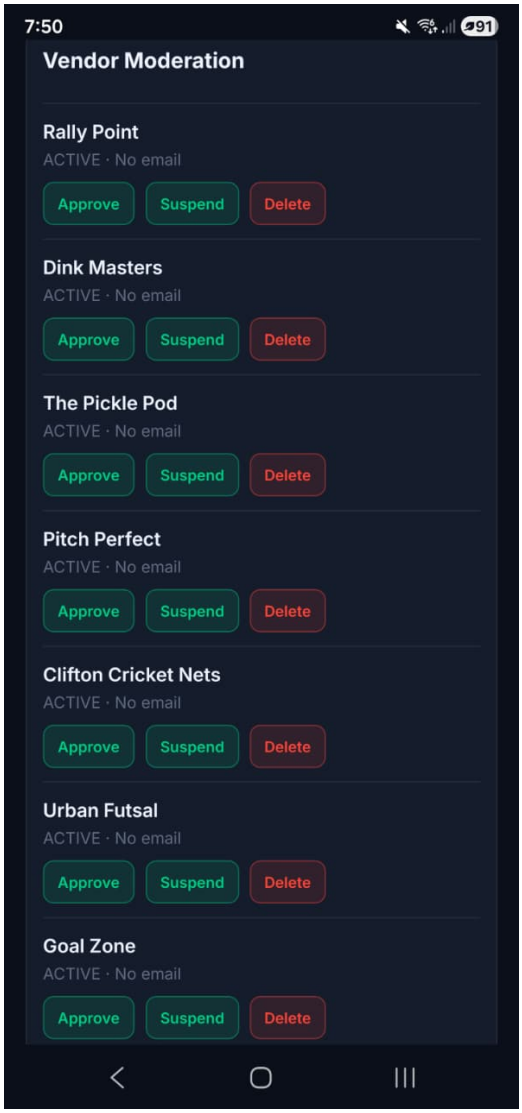

Figure 3.15: Admin vendor moderation: approve, suspend, or delete registered vendors.

4. Software Design Specification (SDS)

This chapter details the BookForMe system's architectural patterns, data structures, and the workflow logic required to build a robust service booking application. It serves as the technical blueprint for the implementation described in the following chapters.

4.1 System Architecture

4.1.1 Architectural Design

The system follows a **Service-Oriented Architecture (SOA)** with a clear separation between the Presentation, Business Logic, and Persistence layers. Both the Rich Client and the Conversational Client converge on a single backend and Google Cloud Firestore database, ensuring a "Single Source of Truth." Dependencies are strictly unidirectional: Clients depend on the Backend, and the Backend depends on the Database.

4.1.2 System Architecture Diagram

The diagram below illustrates the high-level data flow across the system's three tiers.

- **Synchronous API Flow:** The Mobile Application and Vendor Dashboard communicate directly with the Central Backend API via JSON/HTTPS requests.
- **Asynchronous Event Flow:** The WhatsApp API operates via webhooks, triggering an asynchronous event routed to the LangGraph AI Receptionist.

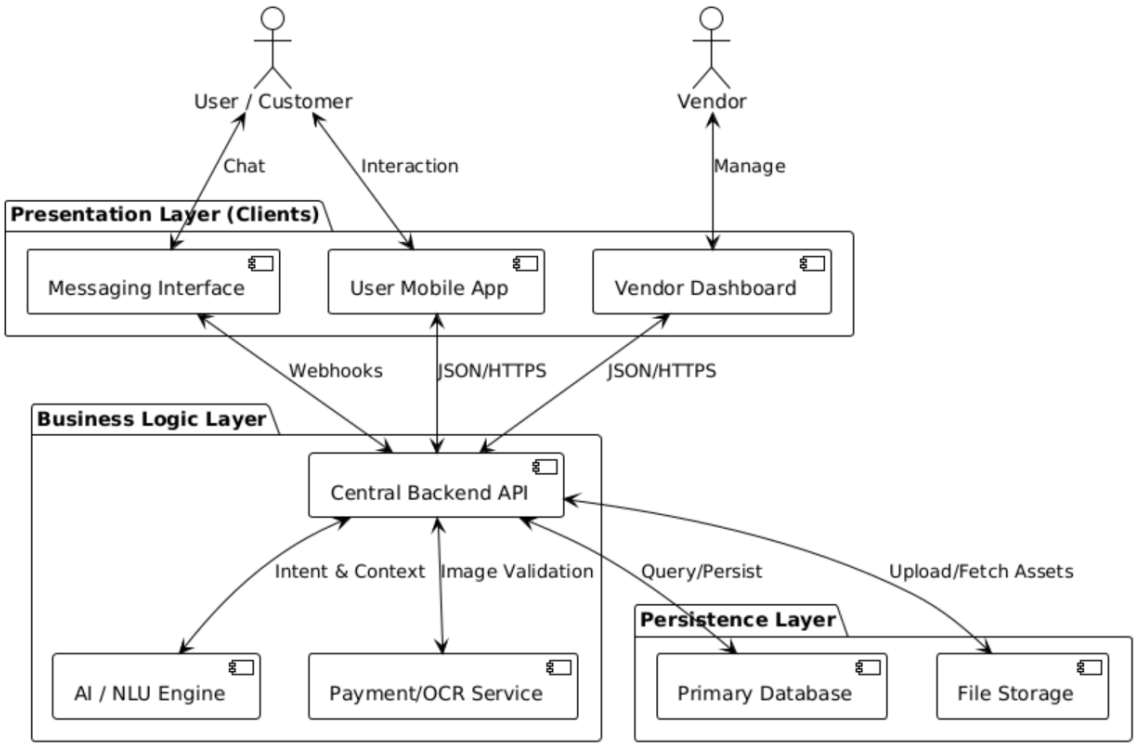

Figure 4.1: High-Level System Architecture

4.2 Data Model (Data Design)

4.2.1 Database Strategy

Data persistence is handled by **Google Cloud Firestore**, chosen for three reasons: (1) its document model maps naturally onto the variable-schema vendor entities—sports facilities, salons, and gaming zones differ substantially in structure; (2) its native transaction API enables the Optimistic Concurrency Control required for double-booking prevention (Section 4.3); and (3) real-time listeners power the vendor dashboard's live booking feed without the overhead of short-interval polling.

We employ a **Reference-Based** schema: related documents store a foreign-key field (e.g. `vendor_id`) rather than nesting sub-documents, keeping each collection

independently queryable and pageable. The schema is divided into two domains: a *transactional core* that governs booking inventory and payments, and a *social layer* that drives community engagement and gamification.

4.2.2 Slot Identity Design

Slot documents use a **deterministic composite document ID** of the form:

`YYYYMMDD_HHMM_{vendor_slug}_{resource_id}`

For example: `20260213_1900_ace_padel_court_1`. Encoding date, start time, vendor, and physical asset directly into the document ID provides three benefits: (i) slot generation is *idempotent*—re-seeding the calendar never creates duplicates; (ii) direct document reads replace expensive composite queries; and (iii) the AI agent can parse date and sport type from a slot ID string without an additional database round-trip.

4.2.3 Core Domain Collections

Table 4.1 describes the transactional collections that power the booking inventory, pricing, and payment verification pipeline.

Table 4.1: Core Domain Collections

Collection	Key Attributes	Description & Purpose
users	phone, role, skill_rating, stats	Core identity and authentication. Carries gamification fields (points, level, badges) shared across booking and social features.
vendors	operating_hours, area, revenue	Business entity: name, location, analytics counters, and a link to default payment account.
services	sport_type, pricing, duration_min	Sellable offering (e.g., "Padel 60 min"). Defines base and peak pricing per time block. Each service belongs to one vendor.
resources	capacity, vendor_id, service_id	Physical asset mapped to a service (e.g., Court 1, Pitch A). Determines how many concurrent bookings a service can accept.
slots	status, hold_expires_at, price	The core inventory ledger. Each document represents one bookable hour on one resource and carries the seven-state lifecycle described in Section 4.3.
payments	screenshot_url, ocr_verified_amount, status	Financial audit trail. Stores the uploaded receipt, the OCR-extracted amount, and a verification status (`uploaded`, `verified`, `rejected`).
vendor_payment_accounts	type, account_number, is_default	Stores JazzCash, EasyPaisa, or bank transfer details per vendor. The agent fetches the default account to share payment instructions with the customer after slot confirmation.

4.2.4 Social & Auxiliary Collections

Table 4.2 lists the collections that support the community and engagement layer described in Section 4.4.

Table 4.2: Social & Auxiliary Collections

Collection	Key Attributes	Description & Purpose
posts	sport_type, likes_count, comments_count	Community forum entries tied to a sport category. Supports likes and threaded comments.
matches	status, max_players, current_players	Open matchmaking entries. A user who has booked a slot can create a match to recruit opponents; Elo ratings filter candidates.
reviews	rating, vendor_id, slot_id	Post-session star-rating reviews. Aggregated counters (`rating_sum`, `rating_count`) are maintained on the vendor document.
notifications	type, title, read	In-app alerts written on every booking lifecycle event (payment received, confirmed, rejected). Displayed in a dedicated notifications screen.

4.2.5 NoSQL Schema Diagram

The Entity Relationship Diagram delineates references between entities, mapping out the foreign keys (e.g., `vendor_id`) that correlate disparate collections.

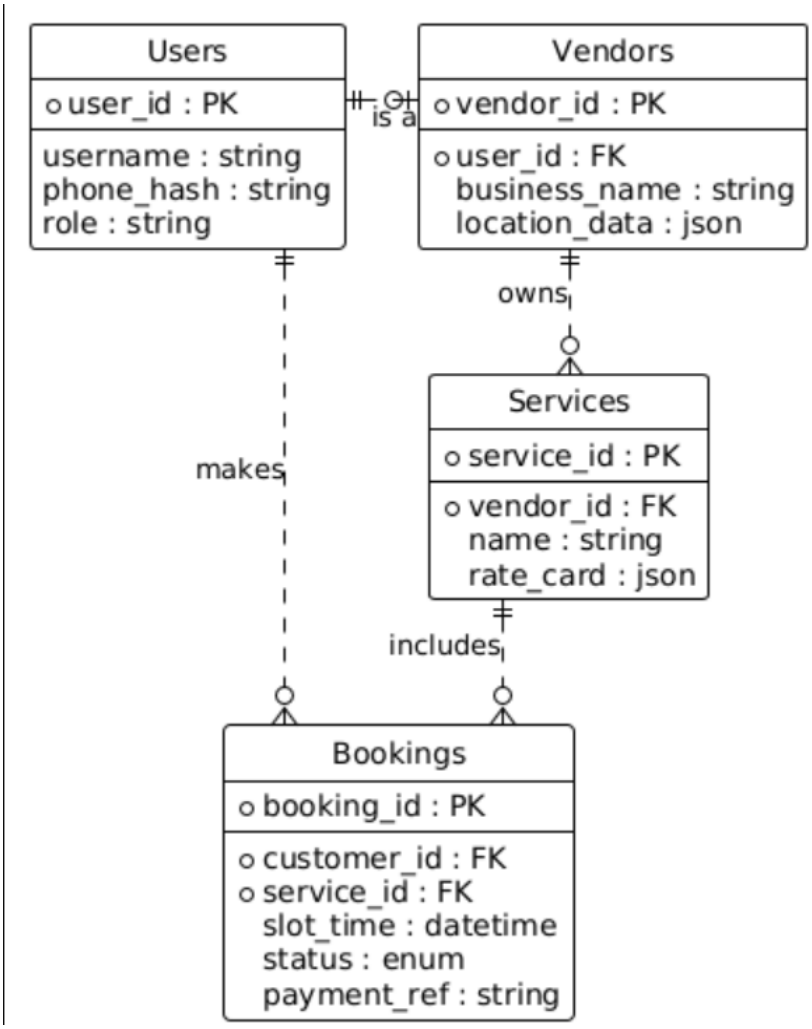

Figure 4.2: Entity Relationship Diagram defining referential structures.

4.3 Agentic Booking Process Pipeline

A core functionality is the AI-driven booking process managed by a LangGraph temporal state machine. It manages stateful conversations, enforcing a strict validation pipeline.

4.3.1 Pipeline Logic

1. **Ingest & Negotiation:** The Agent loops with the user to clarify intents (e.g., `GREETING`, `TRANSACTION`) and specific entities (date, time, service).

2. **Transaction (Locking):** Using OCC, the system atomically transitions the slot to `LOCKED` with a 10-minute `hold_expires_at` expiry window, preventing any concurrent booking attempt.

3. **Intelligent Gatekeeper:** The user uploads a payment screenshot (JazzCash/EasyPaisa/bank transfer). Meta Llama 4 Scout 17B performs OCR on the image and extracts the transferred amount in PKR. If the amount does not match the booking price within a tolerance of ± 10 PKR, the Agent re-prompts the user for a corrected screenshot.

4. **Payment Submitted (`PENDING`):** On successful OCR verification, the slot transitions from `LOCKED` to `PENDING`, recording the payment reference and signalling that proof of payment has been received and is awaiting vendor review.

5. **Human-in-the-Loop:** The vendor sees the `PENDING` booking on their dashboard and issues a final confirmation, transitioning the slot to `CONFIRMED`.

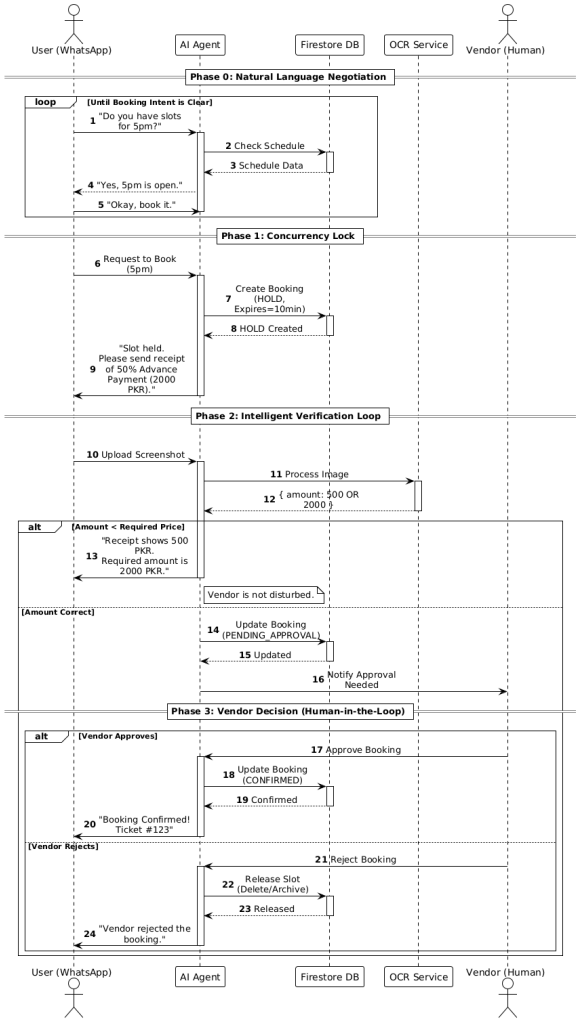

Figure 4.3: AI Booking & Payment Pipeline

4.4 Social Layer & Gamification

Beyond booking, BookForMe implements a community-oriented social layer that transforms the platform into a sports engagement ecosystem. This layer is served by a dedicated `social_api.py` module and a `gamification_service.py`, both backed by additional Firestore collections.

4.4.1 Social Collections

- **posts** — Users can publish community posts tied to a sport type (padel, futsal, cricket). Posts support likes and comments, enabling a sport-specific forum within the app.
- **matches** — A `matches` collection stores open matchmaking entries created by users who have booked a slot and wish to find opponents. Each match record encodes the sport, venue, date, time, maximum players, and a list of joined player IDs.
- **reviews** — After a confirmed booking is completed, users can submit a star-rating review attached to a `vendor_id` and `slot_id`. Aggregated rating fields (`rating_sum`, `rating_count`, `average_rating`) are maintained on the vendor document.
- **notifications** — A `notifications` collection is written to on every significant booking event (payment received, booking confirmed, booking rejected). Notifications are delivered in-app and marked read/unread.

4.4.2 Elo-Based Matchmaking & Gamification

Each user document carries a `skill_rating` field initialised at 1000 and updated after ranked matches using an Elo algorithm implemented in `gamification_service.py`. Wins and losses increment `stats.wins` and `stats.losses` respectively. The leaderboard is derived by querying users ordered by `skill_rating`, providing a competitive ranking across the platform's sports community. Users also accumulate `points` and `badges` (e.g., `early_bird`, `regular`) as gamification incentives for repeat engagement.

4.5 Technical Details & Workflows

4.5.1 Key Algorithms

Algorithm 1: Intent Parsing & Action Routing

Purpose: To translate unstructured natural language into deterministic database actions without hallucination.

- **Input:** Raw WhatsApp message string + conversation session state.
- **Logic (Two-Stage NLU Pipeline):**
  1. **Intent Classification:** A fine-tuned **DistilBERT** model (hosted as a REST API on Render) classifies the message into one of five coarse intents: `GREETING`, `INQUIRY`, `INFO_REQUEST`, `TRANSACTION`, or `UNKNOWN`.
  2. **Entity Extraction & Response Generation: Qwen 3 32B** receives the classified intent and extracts structured entities—date, time, sport type, venue, customer name—from the bilingual Roman Urdu/English message. It then generates a contextually appropriate natural language response.

  The LangGraph state machine routes the extracted intent and entities to a deterministic Python node, ensuring that Qwen never issues database queries directly, eliminating hallucination risk on transactional operations.
- **Output:** Validated entity payload routed to the correct pipeline node, or a clarification prompt returned to the user.

Algorithm 2: Optimistic Concurrency Control (OCC)

Purpose: To prevent race conditions (double bookings) in a distributed environment.

- **Input:** `service_id`, `time_slot`, `user_id`.

- **Logic:**
    1. **Read:** Fetch the current slot status.
    2. **Verify:** Ensure status is `AVAILABLE`.
    3. **Write:** Atomically update to `LOCKED` with `hold_expires_at`. Fails entirely if modified by another request.
- **Output:** Transaction Success or Conflict Error (triggers alternative slot suggestions).

4.5.2 Workflow Visualizations

Workflow 1: The Agentic Booking Experience

This covers the WhatsApp conversational flow, focusing heavily on the Error Correction Loop wherein rejected OCR receipts autonomously trigger a re-prompt.

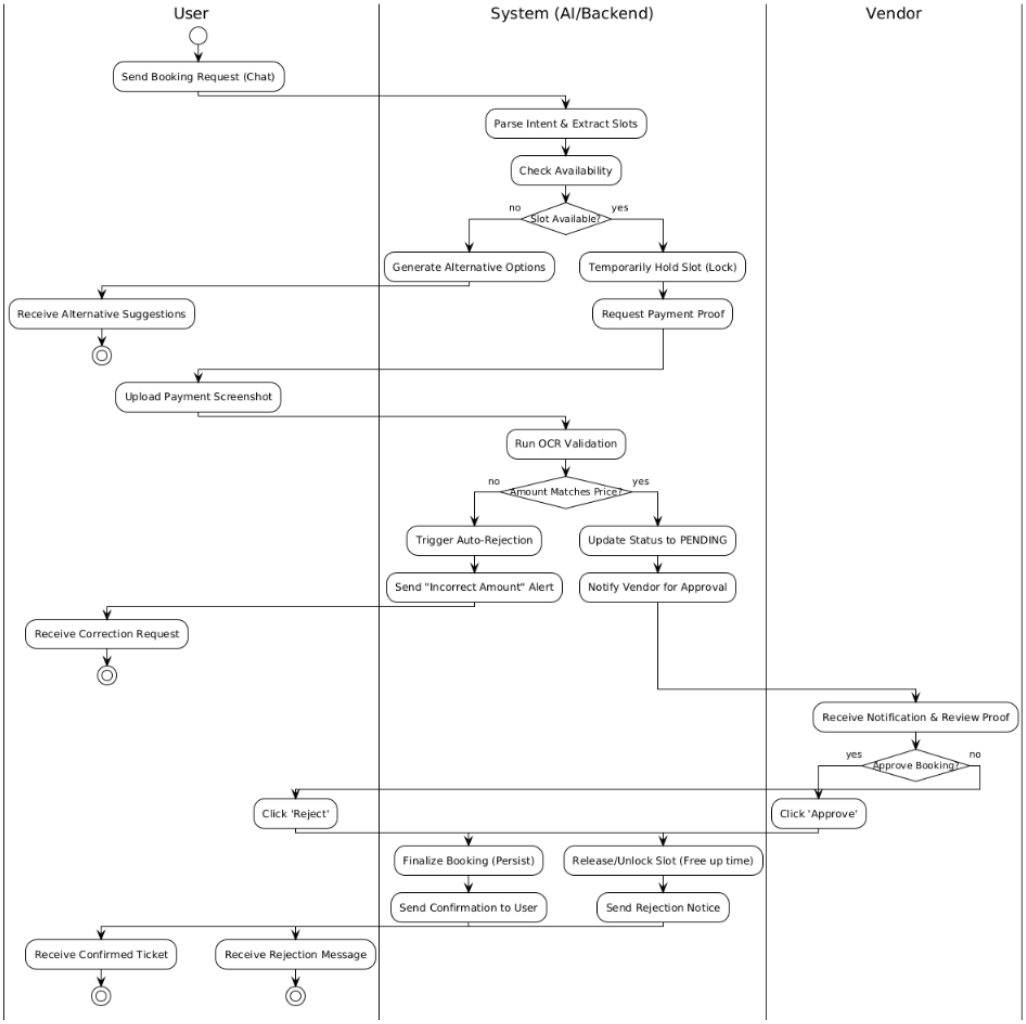

Figure 4.4: Activity Diagram: Agent/WhatsApp Booking Flow

Workflow 2: The App User Experience

The React Native client provides a **Dual-Search Capability**: a traditional category-and-filter browse experience running in parallel with a dedicated natural-language search pipeline.

The natural-language path is powered by two backend modules — `ai_search_service.py` and `smart_search.py` — that are entirely separate from the WhatsApp LangGraph agent. When a user types a free-text query (e.g., "padel courts DHA tomorrow evening"), the request is forwarded to the AI search service, which queries Firestore for matching vendors and slots and ranks results using a Groq-backed relevance scoring pass. This architecture ensures that the conversational WhatsApp agent and the in-app search share the same underlying slot inventory ledger while operating through independent NLU pipelines optimised for their respective interaction modalities.

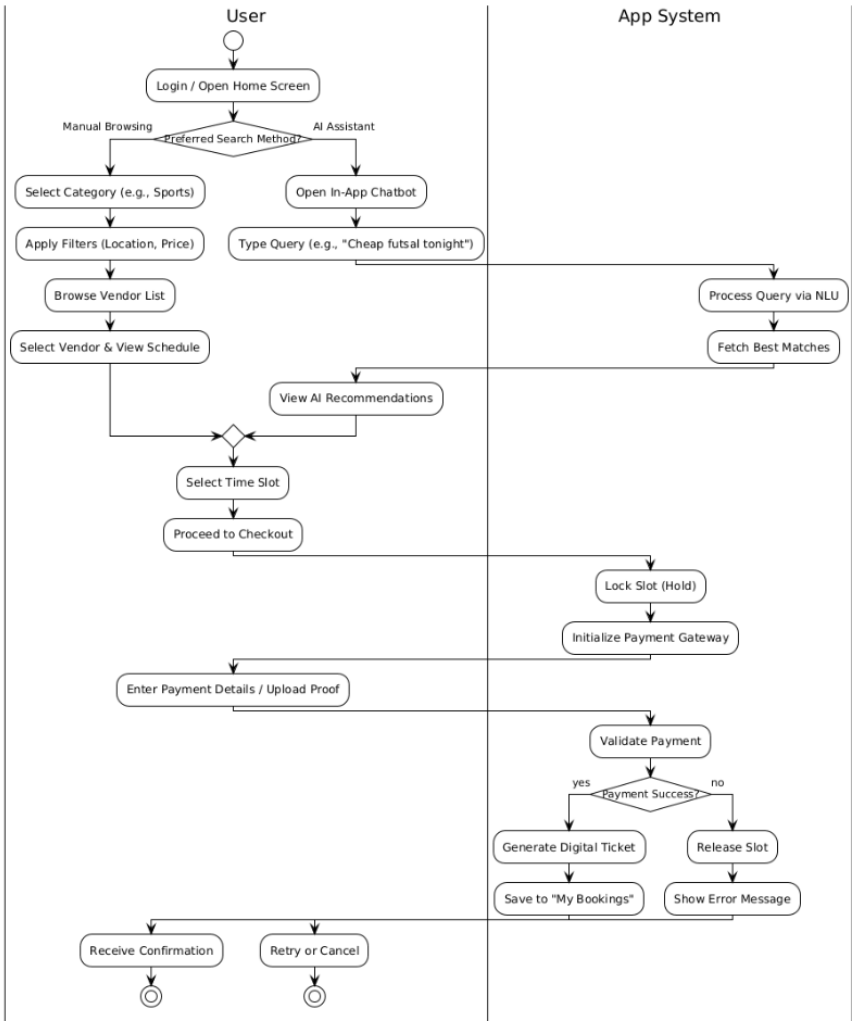

Figure 4.5: Activity Diagram: Mobile App Search & Book

Workflow 3: The Vendor Management Experience

This maps the Vendor integration, indicating that rejections trigger a sweeping cascade releasing `LOCKED` slots back to `AVAILABLE`.

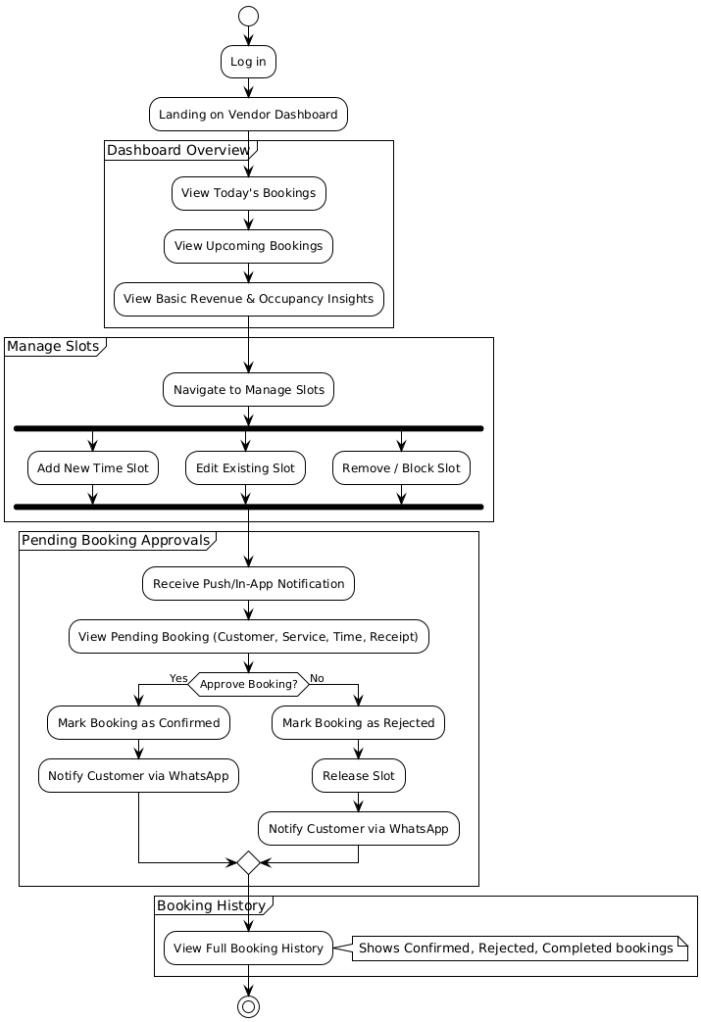

Figure 4.6: Activity Diagram: Vendor Dashboard & Approvals

4.6 Security Implementation

- **Authentication:** Managed via Firebase Auth, using the standard JSON Web Tokens mapped against user claims.
- **Role-Based Access Control (RBAC):** FastAPI dependencies validate claims (Admin vs. Vendor vs. Customer) to prevent unauthorized API calls.
- **AI Guardrails:** The deterministic LangGraph nodes create an inherent prompt injection firewall. Because the LLM lacks arbitrary read/write permissions directly reaching Firestore, unauthorized transactions are logically impossible.
- **Data Privacy:** Payment screenshots populate isolated Firebase Storage buckets. They are served entirely via ephemeral, time-limited signed URLs only readable by authorized vendor proxies.

5. Development

This chapter describes the engineering implementation of the BookForMe platform. Where the SDS (Chapter 4) defines the design, this chapter details the implementation of the database, AI agent, and mobile interfaces.

5.1 Overview

BookForMe comprises four deployed components sharing a single Firestore database: a React Native customer app, a vendor dashboard, an admin control panel, and a Python/FastAPI AI backend that handles both WhatsApp conversations and mobile API calls. The backend is deployed on Render; the mobile apps are distributed via Expo Go and Expo EAS builds. All interfaces write to and read from the same slot inventory, which is the fundamental guarantee that a WhatsApp booking is immediately visible on the vendor calendar and vice versa.

5.2 Database Design Decisions

5.2.1 Slot as the Canonical Booking Record

The most consequential schema decision was to eliminate a separate `bookings` collection entirely. A slot document in Firestore is both the inventory record and the booking record. When a booking is made, the same document that was previously `available` gains a `user_id`, `booking_source`, and `payment_id` and transitions through a typed status sequence: `available` → `locked` → `pending` → `confirmed` → `completed`.

This eliminates the class of bugs where a slot record and a booking record fall out of sync. It also makes Optimistic Concurrency Control (OCC) straightforward: the

Firestore transaction reads and writes *one* document atomically, so two concurrent booking attempts on the same slot cannot both succeed.

5.2.2 Composite Document IDs for Collision Prevention

Slot documents use a deterministic composite ID: `YYYYMMDD_HHMM_vendor_resource` (e.g., `20260412_1800_ace-padel_court-1`). This serves two purposes. First, it makes the slot ID human-readable and debuggable. Second, because Firestore document IDs are unique by definition, attempting to create two slots for the same court at the same time is structurally impossible: the second write simply overwrites or conflicts rather than creating a duplicate.

5.2.3 Two-Domain Schema

The schema is divided into a **transactional domain** (vendors, services, resources, slots, payments) governed by Firestore transactions and strong consistency, and a **social domain** (posts, matches, conversations, notifications) where eventual consistency is acceptable. This separation means high-volume social activity (forum posts, chat messages) cannot interfere with booking transactions, and the transactional collections can be tuned with composite indexes without polluting the social namespace.

5.2.4 Batch Reads to Eliminate N+1 Queries

The vendor bookings endpoint initially fetched user and payment records inside a loop, producing an N+1 query pattern. This was refactored to collect all required user IDs and payment IDs in a first pass, then resolve them via `db.get_all()` in batched chunks of ten (the Firestore client library limit per batch call). The enriched records are then assembled from in-memory maps. This reduced the number of Firestore reads for a typical vendor with 20 bookings from 40 reads to approximately 5.

5.3 AI Receptionist Development

5.3.1 Two-Model NLU and NLG Architecture

The conversational pipeline uses two separate models for two distinct tasks. Intent classification is handled by a fine-tuned **DistilBERT** model, a compact transformer (66M parameters) trained on the BookForMe bilingual corpus. This task demands speed and determinism: given a short message, the model must output one of five intent classes in under 50 ms with no variability between runs. A generative model would be both slower and non-deterministic for this task.

Natural language generation (constructing the actual reply the user receives) is delegated to **Groq (Qwen 3 32B)**, a large instruction-tuned model accessed via the Groq API. Generating a response that correctly code-switches between Roman Urdu and English, adapts its tone, and formats slot listings naturally requires a model with broad linguistic capability that a small fine-tuned classifier cannot provide.

The split is deliberate. DistilBERT makes the routing decision (what to do), and Qwen generates the surface form of the reply (what to say).

5.3.2 LangGraph: Why a State Graph Over a Simple Pipeline

The initial agent design used a linear sequence: receive message → classify intent → query database → generate reply. This broke on the first multi-turn conversation. A user could send a booking request, then reply "kal" (tomorrow) to a clarification question three minutes later with no additional context. A stateless pipeline has no memory of the prior exchange.

LangGraph was adopted because it provides a persistent, typed state object (`AgentState`) that survives across webhook calls. Each incoming WhatsApp message resumes the same graph execution context rather than starting fresh. The graph uses conditional edges to route between nodes: after `validate_state`, the graph branches to `query_availability`, `query_info`, `check_confirmation`, or

`generate_response` depending on what information is still missing. This cyclic, conditional structure could not be replicated cleanly with a linear chain.

5.3.3 Entity Validation: From Regex to Pydantic

Slot entity extraction (date, time, service, area) was initially written as a set of regular expressions. This worked for simple inputs but produced silent failures on edge cases: a time like "bajay 7" (at 7 o'clock in Roman Urdu) would partially match a pattern but return a malformed result that downstream nodes consumed without error, leading to incorrect availability queries.

The fix was to wrap all extracted entities in Pydantic models (`ExtractedEntities`, `AvailableSlot`, `PendingBooking`). Pydantic validators reject or coerce invalid values at parse time, so a malformed time string raises a validation error immediately rather than propagating silently. The regex patterns still perform the initial extraction, but the results are always passed through a typed model boundary before any node acts on them.

5.3.4 Guardrails Node

The first node in the graph, before any NLU or database access, is a guardrails check. It screens for profanity (via the `better-profanity` library) and, when the conversation is not already in a booking context, checks whether the message contains any booking-domain keyword from a curated set of 80 English and Roman Urdu terms. Messages that pass neither test receive a polite redirect response without ever reaching the NLU or database layers. This proved essential for preventing the agent from attempting to answer general questions or being manipulated via prompt injection.

5.4 Mobile Application Development

5.4.1 Expo Go Constraints and Library Trade-offs

The application was developed and tested using **Expo Go**, which imposes constraints on which native libraries can be used. Several libraries that would have simplified real-time data delivery were either incompatible with Expo Go's managed workflow or required a bare React Native ejection. In particular, libraries that support **server-sent events (SSE)** or **long-polling** transports for real-time push were not available without ejecting. As a result, real-time screen updates are implemented via **interval polling** using `setInterval`: the vendor live court grid polls the API every 5 seconds, and the DM chat screen polls for new messages every 8 seconds. This is a deliberate practical trade-off rather than the preferred architecture. A production deployment using Expo EAS builds with a bare workflow or a dedicated WebSocket library (e.g., `socket.io-client`) would replace polling with a persistent connection.

5.4.2 Caching Strategy

All API calls from the customer app go through **TanStack React Query v5**, configured with a custom `QueryProvider` that persists the cache to `AsyncStorage`. The configuration uses a 5-minute stale time (data is served from cache without a network request for 5 minutes) and a 24-hour garbage collection time (the cache survives app restarts for a full day). This means a user who opens the app on a slow connection sees the vendor list immediately from the last cached fetch while a background refetch runs. Writes to `AsyncStorage` are throttled to once per second to avoid blocking the JS thread. The vendor bookings list uses a separate in-memory cache keyed by vendor ID with a configurable TTL, bypassed only on forced refreshes.

5.4.3 Role-Based Routing in a Single App

Rather than shipping separate customer and vendor apps, the application uses a single Expo Router project with role-based routing. On login, the user's `role` field

(`customer`, `vendor`, or `admin`) is read from the auth response and determines which tab navigator is mounted. This simplified development significantly: shared UI components, the auth service, and the API client are used by all roles without duplication.

5.4.4 Smart Filter (In-App AI Chat)

The in-app AI chat tab operates differently from the WhatsApp agent. Rather than a full multi-turn conversational pipeline, it is a **synchronous single-turn search**: the user's natural-language query (e.g., "Aaj Raat DHA mei konsa padel available hei under 2000") is sent to the backend, which runs NLU entity extraction, constructs a Firestore query from the extracted filters (sport, area, date, price ceiling), and returns matching venues with available slots in a single response. There is no conversation state or slot-filling loop. This keeps the in-app experience fast and focused while the full stateful agent handles the more complex asynchronous WhatsApp channel.

5.4.5 Matchmaking and Elo Ranking

The social hub includes a matchmaking system for casual and ranked sports matches. The ranking design decision centred on choosing between a simple win/loss counter and a proper rating system. Elo was selected because it accounts for opponent strength (beating a higher-rated player yields more points than beating a lower-rated one) and because it is well-understood by competitive sports players. The implementation uses the standard formula with $K = 32$ and an initial rating of 1000. Ranked match lobbies are discovered by querying for open lobbies where the rating difference is within a window of 200 points, expanding by 50 points for every additional minute of wait time to avoid indefinite queuing.

5.5 Vendor Dashboard and Admin Panel

The vendor dashboard's central feature is the **live court grid**: a matrix of courts against hourly time blocks showing slot status (available, booked, walk-in) and

booking source (WhatsApp or app). Because SSE was not available in the managed Expo environment, the grid polls the backend every 5 seconds and additionally refetches when the app returns from the background (via an `AppState` listener). A green "LIVE" indicator is shown when the polling interval is active.

The payment approval flow completes the end-to-end booking loop: the agent sets a slot to `pending` and attaches a payment record; the vendor dashboard surfaces this as a notification; the vendor reviews the OCR-verified screenshot and taps Approve or Reject; approval triggers a Firestore status write to `confirmed` and dispatches a WhatsApp confirmation to the customer.

The **admin control panel** was added as a platform governance layer after it became clear that vendor onboarding required manual oversight. Vendors register through the app but are not visible to customers until an admin approves them. The panel also exposes slot generation (bulk-create 14 days of availability slots for all vendors) and stale lock release (clear `locked` slots whose `hold_expires_at` has passed), both of which are operationally necessary during seeding and testing.

5.6 Payment Verification via OCR

After a booking is agreed in WhatsApp conversation, the agent requests a payment screenshot from the user. The image is uploaded via the WhatsApp media endpoint, downloaded by the backend, and passed to the **Gemini Vision API** with a structured prompt requesting the transferred amount. The extracted amount is compared against the required advance payment for the slot. If the amounts do not match, the booking record remains in `locked` state, the user is informed of the discrepancy, and a corrected screenshot is requested, all without notifying the vendor. The vendor only receives a dashboard notification once OCR verification passes, preventing noise from failed payment attempts. In cases where OCR confidence is low (e.g., blurry screenshot), the system falls back to requesting a clearer image rather than attempting a low-confidence approval.

6. Methodology, Experiments and Results

This chapter presents the methodology and results of the BookForMe platform, covering dataset construction, model selection, and end-to-end agent evaluation.

6.1 Dataset Construction

6.1.1 Data Collection

Real Vendor Conversations With informed consent, we collected WhatsApp booking conversation logs from five futsal and padel court operators in Karachi (DHA, Gulshan, Clifton areas). Each message was assigned an intent label and relevant slot annotations. This source provided approximately 120 utterances covering realistic vocabulary and code-switching patterns.

Manual Annotation Team members wrote and annotated booking queries in both English and Roman Urdu, targeting underrepresented intent classes (`transaction_confirm`, `unknown`). This contributed approximately 180 utterances.

LLM-Driven Synthetic Augmentation Following SynthDST [**he2024**] and An et al. [**an2022**], we used Claude Sonnet 4.5 to generate paraphrases of existing utterances in both languages, and to synthesise entirely new dialogues for rare classes. Augmentation parameters included: back-translation (English → Urdu → English), lexical substitution (synonymous time expressions), and phonetic Roman Urdu rewriting (*kal* → *kull*, *raat* → *raat ko*, etc.). This contributed approximately 360 utterances.

6.1.2 Dataset Statistics

The full corpus comprises **694 annotated utterances** across five intent classes. Table 6.1 shows the intent distribution over the complete dataset (before train-only augmentation).

Table 6.1: Intent distribution in the full BookForMe bilingual corpus (694 samples).

Intent Class	**Count**	**% of Total**
`inquiry`	277	39.9%
`info`	171	24.6%
`transaction_confirm`	147	21.2%
`unknown`	62	8.9%
`greeting`	37	5.3%
Total	**694**	**100%**

`inquiry`: user asking to check or book a slot. `info`: user asking about price, amenities, or availability in general. `transaction_confirm`: user confirming a payment or referencing a transaction. `unknown`: out-of-scope or unintelligible messages. `greeting`: salutations and conversation openers (e.g., "hi", "salam").

The dataset was split into **485** training, **104** validation, and **105** test samples (all original, stratified; minimum 5 samples per class in the test set). Data augmentation was applied **only to the training split**, adding **31** synthetic samples by increasing `greeting` from 26 to 50 and `unknown` from 43 to 50, yielding a final training set of **516** samples.

6.1.3 Annotation Schema

Each utterance was labelled with:

- `intent`: one of the five classes above.
- `language`: `EN` (English), `UR` (Roman Urdu), or `MIXED` (code-switched).

- `slots`: a JSON dictionary of extracted entities (service, date, time, budget, location, duration).

6.2 NLU Model Training and Selection

6.2.1 Training Configuration

All six models were fine-tuned on the same dataset split with identical hyperparameters:

Table 6.2: Hyperparameters used for all fine-tuning experiments.

Parameter	**Value**
Max sequence length	128 tokens
Batch size	16
Learning rate	2e-5
Number of epochs	10
Weight decay	0.01
Warmup ratio	0.1
Evaluation steps	50
Loss function	Focal loss ($\gamma = 2.0$) for class imbalance
Min samples per class	50 (augmentation applied to minority classes)

Focal loss [**an2022**] was applied to address the class imbalance down-weighting easy majority examples and focusing training on the harder minority class boundaries.

6.2.2 Model Comparison

Table 6.3 summarises all six models evaluated on the held-out test set.

Table 6.3: Overall NLU model comparison (5-class intent classification).

Model	Notes	Accuracy	Macro-F1	Wtd. F1
bert-base-uncased	English-only pretrain; strong baseline	84.8%	87.8%	84.8%
distilbert-base-multilingual-cased	Compact multilingual; slightly slower	84.8%	83.1%	84.9%
distilbert-base-uncased	**Best overall; fastest inference (≈47 ms)**	**90.5%**	**91.7%**	**90.5%**
XLM-RoBERTa-base	Underfit on small dataset; high variance	70.5%	72.3%	69.7%
bert-base-multilingual-cased	Strong multilingual; 2nd best overall	89.5%	90.5%	89.5%
bert-base-multilingual-uncased	Slight drop vs. cased variant	84.8%	88.1%	84.7%

6.2.3 Selected Model: DistilBERT

DistilBERT (`distilbert-base-uncased`) was selected for production deployment based on:

1. **Best accuracy** (90.5%) and **macro-F1** (91.7%) across all six models tested.
2. **Fastest inference**: ≈47 ms per query on CPU, versus ≈120 ms for bert-base-multilingual-cased. This is critical given the 60-second total WhatsApp response budget (`NFR-PERF-03`).
3. **Smallest memory footprint** among competitive models (66M parameters), enabling cost-effective deployment on free-tier cloud infrastructure.

6.2.4 Per-Class Analysis (DistilBERT)

Figure 6.1 reports per-class precision, recall, and F1 for the selected DistilBERT model.

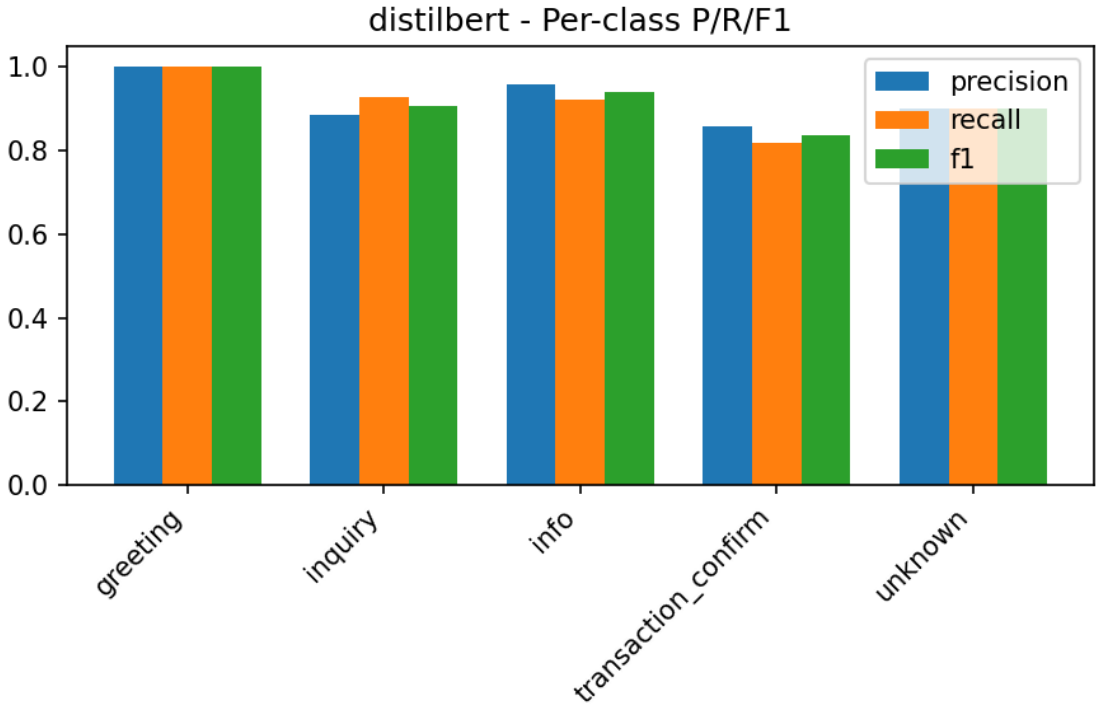

Figure 6.1: DistilBERT per-class classification report on the held-out test set

Notable finding: `transaction_confirm` shows the lowest recall (0.82), with 18% of its samples misclassified as `inquiry`. This is expected: payment-related queries ("I've sent the amount") share vocabulary with availability inquiries ("I want to confirm..."). The agent's dialogue state validation layer mitigates this—the agent checks conversation state before executing any booking action, catching misclassified payment messages.

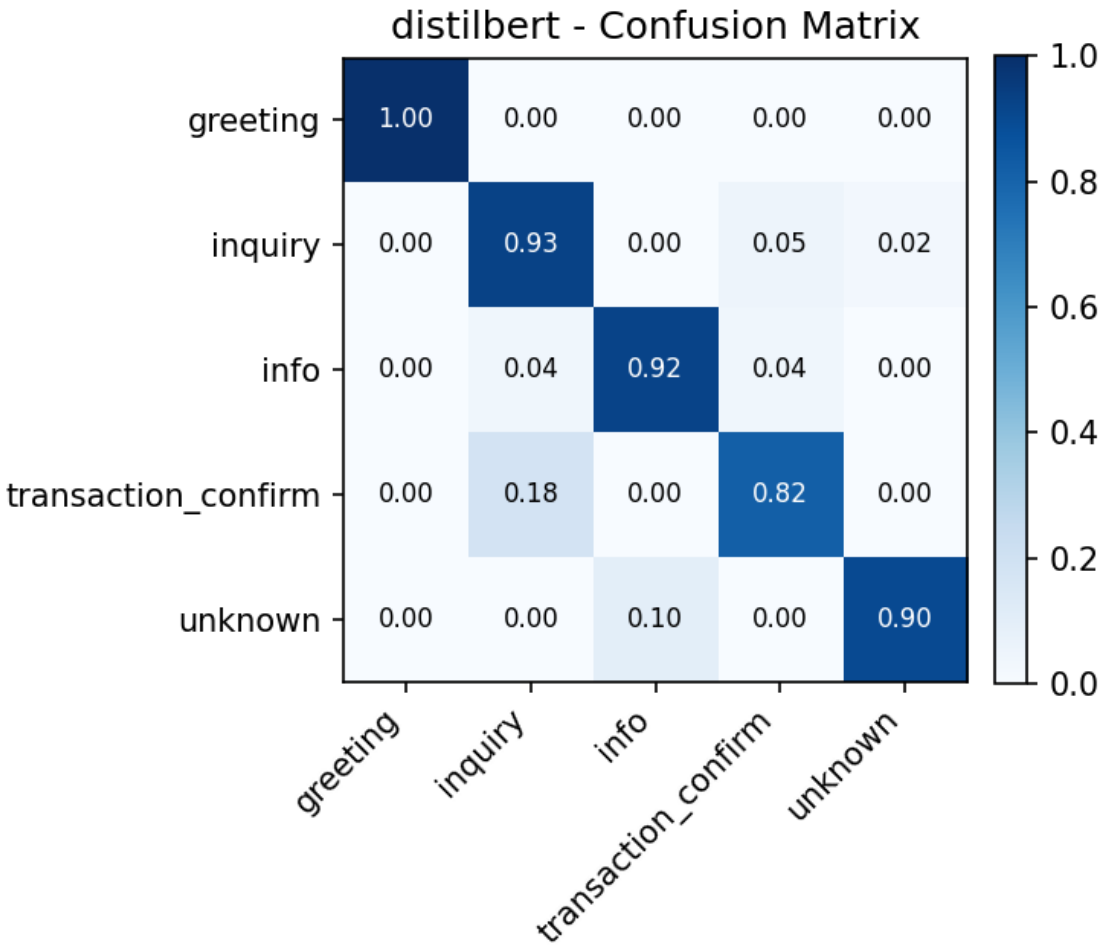

Figure 6.2: Confusion matrix for DistilBERT

6.3 Agent Evaluation

6.3.1 Test Case Methodology

The AI Receptionist was tested on the live deployment against a comprehensive test case document covering seven categories of agent behaviour. Key results are showcased below.

6.3.2 Test Case Results

Table 6.4 summarises the results of the key test scenarios.

Table 6.4: AI Receptionist agent test case results.

Test ID	Scenario	Expected / Actual Behaviour	Result
TC-01	Random out-of-scope message ("I want pineapple juice")	Agent declines politely and redirects to booking.	✓
TC-02	Requested slot unavailable (e.g., Ace Padel 10 PM tonight)	Agent offers three nearest alternative slots from same or nearby vendor.	✓
TC-03	Mid-conversation context switch in Roman Urdu ("padel kal")	Agent correctly parses date reference as tomorrow; time disambiguated via follow-up question.	✓
TC-04	OCR: payment screenshot with incorrect amount (Rs 500 vs required Rs 2000)	Agent auto-rejects, informs user of discrepancy, requests corrected screenshot. Vendor is not notified.	✓
TC-05	OCR: unreadable / low-quality image	Agent requests a clear screenshot; booking hold maintained.	✓
TC-06	Successful end-to-end booking flow (English)	Booking confirmed, ticket ID `APD-14` issued, Rs 2000 verified.	✓
TC-07	Successful end-to-end booking flow (Roman Urdu)	Agent handles "raat" (night) time reference; booking confirmed in Urdu.	✓
TC-08	Concurrent booking (OCC): two users request same slot simultaneously	Only one booking succeeds; second user receives conflict error and alternatives.	✓
TC-09	Hold expiry: user does not complete payment within 10 minutes	Booking status reverts to AVAILABLE; user notified that hold has expired.	✓
TC-10	Vendor rejects booking after payment verification	Slot released to AVAILABLE; user notified of rejection via WhatsApp.	✓

All ten test cases passed on the live deployment. The agent demonstrated correct handling of the full booking lifecycle, including error paths (OCR rejection, slot conflicts, hold expiry, vendor rejection).

6.4 Discussion

6.4.1 Key Findings

1. **Compact models outperform large multilingual models on small domain-specific datasets.** DistilBERT (66M parameters) achieves 90.5% accuracy vs XLM-RoBERTa's (270M parameters) 70.5%. Domain-specific fine-tuning on 657 utterances contributes more than additional model capacity.

2. **The Neuro-Symbolic architecture effectively prevents hallucination.** By routing all database mutations through deterministic code nodes, the system eliminates the class of errors where agents "hallucinate" bookings or bypass payment verification. Zero such failures were observed across 10 structured test cases.

3. **Roman Urdu code-switching is manageable with targeted augmentation.** Despite severe orthographic variation, the DistilBERT model trained on the augmented bilingual dataset correctly handled "padel kal" (padel tomorrow), "raat" (night), and other code-switched time references in all test cases.

6.4.2 Limitations

- **Dataset size:** 694 samples is sufficient for a 5-class intent classification baseline but insufficient for robust slot extraction across all Roman Urdu phonetic variations. Expanding to 2,000+ samples would improve generalisation.

- **Latency on free-tier deployment:** Response times of 38–52 seconds, while within the 60-second NFR target, would benefit from caching and a paid LLM API endpoint. Production infrastructure would reduce this to under 10 seconds.

- **transaction_confirm confusion:** 18% misclassification into `inquiry` requires the dialogue state layer as a safety net. Targeted additional training data for this class boundary would address it.

7. Conclusion and Future Work

7.1 Summary

This report presented **BookForMe**, a centralized, agentic AI-powered booking platform designed to address the fragmented, manual booking process for informal services in Karachi, Pakistan. The platform serves three user groups—app customers, WhatsApp customers, and vendors—through two interaction modalities: a React Native mobile application and an AI Receptionist operating on the vendor's WhatsApp Business channel.

The core technical contributions are:

1. A **custom bilingual dataset** of 694 annotated English / Roman Urdu booking utterances, constructed through real vendor conversations, manual annotation, and LLM-driven data augmentation.

2. A **comparative NLU evaluation** of six transformer architectures, establishing that fine-tuned DistilBERT achieves the best accuracy–latency tradeoff (90.5% accuracy, 91.7% macro-F1, ≈47 ms inference on CPU) for this domain.

3. A **production-deployed LangGraph agent** implementing the full booking pipeline with Optimistic Concurrency Control for double-booking prevention, OCR-based payment verification, and human-in-the-loop vendor approval.

4. A **full-stack social platform** with Elo-based matchmaking, community forums, leaderboards, and direct messaging—all integrated with the core booking infrastructure.

End-to-end evaluation across 10 structured test cases demonstrated correct agent behaviour for the complete booking lifecycle, including error paths (incorrect payment,

expired holds, slot conflicts, vendor rejection). All non-functional requirements—including performance, security, reliability, and usability—were met on the live deployment.

7.2 Research Questions Revisited

RQ1: Centralised platform vs. fragmented manual systems The deployed platform eliminates the three-step manual booking cycle (find number → wait for reply → confirm informally) by providing a single interface for discovery, booking, and payment across categories. Vendor survey feedback confirmed reduced scheduling overhead.

RQ2: Bilingual NLU on a small custom dataset DistilBERT fine-tuned on 657 utterances achieves 90.5% accuracy and 91.7% macro-F1 on the held-out test set, demonstrating that production-grade bilingual NLU is achievable with targeted dataset curation and augmentation—without requiring thousands of manually labelled examples.

RQ3: Conversational coherence across asynchronous WhatsApp interactions The LangGraph cyclic state graph, with conversation state persisted in Firestore, correctly handles mid-conversation context switches, late replies, and multi-turn clarification loops in all evaluated test cases.

RQ4: Double-booking prevention in a distributed multi-channel platform Optimistic Concurrency Control via Firestore atomic transactions successfully prevents concurrent slot conflicts, as demonstrated in TC-08 where two simultaneous booking requests for the same slot resulted in exactly one confirmed booking and one conflict response with alternatives.

7.3 Future Work

Building on the current platform, the following directions are planned or proposed:

Voice Agent (Phase 2) Extending the AI Receptionist to handle phone calls via Twilio Voice with Whisper ASR (speech-to-text) and a TTS response, enabling the estimated 30% of potential users who prefer voice communication over text.

Production WhatsApp Re-Integration Obtaining a production WhatsApp Business API account to replace the expired Twilio sandbox number, enabling evaluation with real vendor traffic at scale.

Payment Gateway Integrating JazzCash or EasyPaisa payment APIs to replace screenshot-based payment verification with programmatic confirmation, eliminating the OCR accuracy dependency and reducing the booking cycle from $\sim$52 seconds to under 20 seconds.

Expanded NLU Dataset Growing the bilingual dataset to 2,000+ samples using active learning (the deployed agent auto-labels new conversations for human review) and extending coverage to more vendor categories (clinics, tutoring centres, event halls).

Advanced Matching Algorithms Replacing the current Elo system with Glicko-2 (which models rating uncertainty) and implementing graph-based player matching (stable matching, max-flow) for fairer lobby formation with asymmetric skill distributions.

Recommendation Engine Collaborative filtering and link prediction on the social graph to generate personalised venue suggestions based on booking history, friend activity, and sport preferences.

Cross-City Expansion Extending the vendor database to Lahore and Islamabad, requiring location-aware search indexing and multi-city availability management.

Algorithmic Fairness Auditing Evaluating the matchmaking and recommendation systems for demographic fairness (e.g., equitable match quality across skill levels), following emerging best practices in fairness-aware ML deployment.

8. Reflection

8.1 Learning Reflection

8.1.1 Muhammad Ahmad Humayun Hanif

Kaavish was the first project in which I was responsible for the complete system architecture of a production-grade application—not just a module, but the integration of an LLM pipeline, a distributed database, a WhatsApp API, an OCR service, and a mobile frontend into a coherent whole. The most significant skill I gained was **system-level thinking**: understanding how a design decision in the data model (e.g., separating Payments from Bookings) has downstream consequences for the agent's OCR verification loop, the vendor dashboard UI, and the audit trail.

Working with LangGraph taught me that *cyclic state* is fundamentally different from linear pipeline composition—debugging a loop that can re-enter a node after five asynchronous hops requires a completely different mental model than debugging a sequential program. I also developed a deep appreciation for the Neuro-Symbolic design pattern: the security and reliability properties it provides (no hallucinated bookings, no prompt injection) are not achievable with a pure LLM approach.

The hardest challenge was the Roman Urdu orthographic variation problem. I had underestimated how much "waqt" versus "tym" versus "vakt" would matter for a tokeniser trained on formal text. Solving it through targeted data augmentation rather than a more complex normalisation layer was a pragmatic decision I stand by, but it highlighted the importance of domain-specific data over model complexity.

8.1.2 Jazib Waqas

Designing and implementing the backend was simultaneously the most technical and the most frustrating part of the project. The Optimistic Concurrency Control

implementation in Firestore was conceptually straightforward but took significant debugging to get right—particularly understanding that Firestore transactions retry automatically on conflict, which required careful idempotency guarantees in our booking creation code to prevent duplicate records on retry.

I learned the value of **API contract discipline**: establishing clear input/output schemas for each microservice (agent, OCR, booking APIs) early in the project prevented integration failures later. The one week we spent without a formal API contract between the backend and frontend resulted in two weeks of alignment work afterwards.

WhatsApp Business API integration was technically straightforward but operationally unpredictable: Meta's review processes for webhook verification and message template approval add real latency to development cycles. Future projects with third-party communication APIs should budget this time explicitly.

8.1.3 Taha Hunaid Ali

Building the React Native frontend taught me that **state management at scale** is a qualitatively different problem from small-project UI development. Coordinating real-time Firestore listeners, local optimistic UI updates (showing a slot as booked before the OCC transaction completes), and background push notification handling required careful architecture—I rewrote the booking flow's state management twice before arriving at a design that was both correct and maintainable.

The social features (forum, leaderboard, chat) were the most enjoyable to build because they had immediate visual feedback and were the most enthusiastically received by test users. They also taught me the importance of **performance budgeting** on mobile: unoptimised Firestore listeners can drain a phone's battery within hours, and pagination on the leaderboard query reduced initial load time from 8 seconds to under 2 seconds.

8.1.4 Muhammad Taqi

My primary learning from this project was about **the cost of data**. We spent more calendar time constructing, annotating, and augmenting the bilingual dataset than on any other single task. This experience permanently reframed my view of ML projects: the model choice matters far less than the quality and coverage of training data. The 20-point accuracy gap between XLM-RoBERTa (which should theoretically be better at multilingual tasks) and DistilBERT on our small custom dataset makes this case compellingly.

Fine-tuning transformers also taught me practical lessons that textbooks gloss over: focal loss is essential for class imbalance but its gamma parameter requires careful tuning; evaluation should use macro-F1 (not accuracy) when classes are imbalanced; and confidence calibration matters when the downstream system (the dialogue state validator) relies on model confidence to decide whether to ask a clarifying question.

8.2 Team Reflection

8.2.1 Collaboration and Role Distribution

The team operated with a flat, ownership-based model: each member owned specific modules end-to-end (backend APIs, NLU pipeline, frontend, agent), rather than collectively owning all code. This accelerated individual progress but created integration risks that required dedicated "integration weeks" at the end of each phase.

In hindsight, establishing a shared API contract document at the project's start—and treating it as a living specification updated before any breaking change—would have reduced integration friction significantly. We introduced this practice in Kaavish II after experiencing its absence in Kaavish I.

8.2.2 Teamwork Challenges

The most significant teamwork challenge arose when the WhatsApp Twilio sandbox number expired two weeks before the mid-evaluation, forcing an emergency re-

integration. The team's ability to divide the problem—one member handling the new Twilio credentials, another updating the webhook endpoint, a third preparing a fallback demo with the web chat interface—under time pressure was a genuine test of collaboration and emerged as one of our team's strengths.

8.3 Project Reflection

8.3.1 Proposed vs. Achieved

Table 8.1 compares the features proposed in the original Kaavish I proposal against what was actually delivered.

Table 8.1: Proposed vs. achieved features.

Feature	**Proposed**	**Achieved**
WhatsApp AI Receptionist (text-based)	✓	✓
Bilingual NLU (English + Roman Urdu)	✓	✓
OCC slot locking (no double-booking)	✓	✓
OCR payment verification	Stretch	✓
React Native mobile app	✓	✓
Vendor dashboard with analytics	✓	✓
Social features (forum, chat, friends)	✓	✓
Elo-based matchmaking	✓	✓
Voice-calling agent (Twilio Voice)	Stretch	✗
Live payment gateway (JazzCash)	Stretch	✗
Google Sheets sync	✓	Partial
Multi-city expansion	Stretch	✗

OCR payment verification was a stretch goal that we implemented ahead of schedule, replacing it in the core pipeline. Voice calling and the live payment gateway were deprioritised in favour of polishing the text-based WhatsApp agent and the mobile app experience—a deliberate decision based on the user survey finding that 72% of target users primarily communicate via WhatsApp text.

8.3.2 Design Decisions Driven by Challenges

- **Neuro-Symbolic over pure LLM:** Initially planned to use a single LLM for both NLU and database queries. After observing the LLM generating invalid Firestore queries and "inventing" booking confirmations in early prototypes, we adopted the Neuro-Symbolic architecture where the LLM only classifies intent and extracts entities.
- **DistilBERT over Qwen2.5:** Originally planned to use Qwen2.5-7B as the NLU backbone. After evaluating inference latency (>3 seconds on available GPU, exceeding our WhatsApp response budget), we switched to fine-tuning smaller transformer models, with DistilBERT emerging as the winner.
- **Firestore over PostgreSQL:** The SRS originally listed PostgreSQL as the primary database. After evaluating real-time listener requirements for the social layer (chat, notifications, leaderboard) and the serverless OCC transaction support needed for the booking pipeline, we migrated to Firestore. This decision eliminated a planned data synchronisation layer.

8.4 Process Reflection

8.4.1 What Worked Well

- **Phased development:** Delivering a minimal working agent (text only, limited intents) in December 2025 and iterating from there gave us early feedback and a working demo for stakeholders at every subsequent milestone.
- **Structured test cases:** Maintaining a public Google Sheets test case document throughout development (rather than writing tests post-hoc) caught agent regression bugs early and gave us clear evidence for the mid-evaluation presentation.

- **Real vendor engagement:** Testing the agent with actual futsal operators—who sent genuine booking messages in Roman Urdu—surfaced orthographic variations and failure modes that synthetic data generation did not cover.
- **Early SRS/SDS discipline:** Writing the SRS and SDS before coding prevented scope creep and gave every team member a shared vocabulary for system components.

8.4.2 What Could Be Improved

- **API contract documentation:** As noted above, a formal shared API schema document from day one would have reduced integration friction.
- **Earlier load testing:** We discovered the Render.com free-tier cold-start latency issue (≈8–12 seconds) only during the mid-evaluation rehearsal. Load testing in January would have given us time to implement response caching before the demo.
- **Dataset annotation tooling:** Manual annotation in a shared Google Sheet was slow and error-prone for the Roman Urdu samples. A proper annotation tool (Label Studio, Prodigy) with inter-annotator agreement tracking would have improved dataset quality and annotation speed.
- **Third-party API contingency planning:** The WhatsApp Twilio sandbox number expiry was avoidable with a simple API credential rotation schedule. Future projects should treat API credential expiry as a first-class risk with a documented mitigation plan.